

Computer Science Department
Software Engineering & Business Analysis

Bachelor's Thesis

**Vakansio: A Personalized Job-
Matching Service**

Clean Architecture, LLM-Based Scoring, and Gold-Set
Evaluation on .NET 8

Yan Khadnevich

Supervisor

KSE, Volodymyr Skochko, vskochko@kse.org.ua

Expert

KSE, TBD, expert@kse.org.ua

Submission date

03 June 2026

Academic Integrity Statement

I, undersigned, hereby declare that this capstone project is the result of my own work.

- All ideas, data, figures and text from other authors have been clearly cited and listed in the bibliography.
- No part of this project has been submitted previously for academic credit in this or any other institution.
- All code, diagrams, and third-party materials are either my original work or are used with permission and properly referenced.
- I have not engaged in plagiarism or any form of academic dishonesty.
- Any assistance received (e.g. from peers, tutors, or online forums) is acknowledged in the acknowledgements section.

I understand that failure to comply with these declarations constitutes academic misconduct and may lead to disciplinary action.

Place, date Kyiv, 04.06.2026

Signature

Contents

Acknowledgements	1
Abstract	2
1 Introduction	3
1.1 Initial situation	3
1.2 Motivation	3
1.3 Objectives	4
1.4 Method	5
1.5 Structure of this thesis	5
2 Research	7
2.1 The Ukrainian job-search landscape	7
2.2 Existing job sites and what they offer	7
2.3 Legal and technical constraints on data collection	10
2.4 Related work in recommender systems	12
2.5 LLM-based evaluation frameworks	12
2.6 Gaps and my positioning	13
2.7 Functional and non-functional requirements	14
3 Design	15
3.1 Architectural drivers	15
3.2 Technology stack selection	16
3.3 Clean Architecture layering and SOLID mapping	17
3.4 Scoring pipeline design	19
3.5 Cache hierarchy	22
3.6 Frontend architecture and UX design	22
3.7 Deployment topology	24
3.8 Observability and security design	25
3.9 Chapter summary	27
4 Implementation	28
4.1 Development methodology	28
4.2 Domain layer walkthrough	29
4.3 Application layer walkthrough	31
4.4 Infrastructure layer walkthrough	33
4.5 Frontend implementation details	35
4.6 Deployment, operations, and operating costs	37
4.7 Selected engineering narratives	40
4.8 Chapter summary	42
5 Validation	43
5.1 Evaluation philosophy and methodology pivot	43
5.2 Layered evaluation architecture	44
5.3 Layers 1 and 2 — frozen normalisation baselines	45
5.4 Layer 3 — score as a calibration diagnostic	46
5.5 Layer 4 — reason-text quality and the prompt iteration cycle	47
5.6 Layer 5 — top-ten ranking quality (the primary endpoint)	49

5.7 Threats to validity	51
5.8 Chapter summary	53
6 Conclusion	54
6.1 Project summary	54
6.2 Comparison with the initial objectives	54
6.3 Encountered difficulties	55
6.4 Future perspectives	56
6.5 Final reflection	56
Glossary	58

Acknowledgements

I would like to express my gratitude to my supervisor, **Volodymyr Skochko**, whose methodological guidance shaped both the architectural choices in this project and the way I framed the evaluation chapter. Your feedback turned several rough drafts into the structure the thesis stands on today, and the methodology pivot reported in Chapter 5 would not have landed cleanly without it. Many thanks for your time and your patience throughout the supervision process.

I also owe a great deal to several lecturers at the Kyiv School of Economics whose teaching shaped what I was able to build.

Artem Korotenko, the Academic Director, first showed me what programming actually is. Long before I could think about Clean Architecture or SOLID, his courses gave me the intuition for what a well-structured program even looks like. The discipline that the Domain layer of Vakansio enforces, with no external dependencies and every collaborator behind an interface, is the same discipline I first encountered in his classroom.

Ihor Pysmennyi taught me how production systems actually run. His DevOps course was the first place I saw containers, reverse proxies, and managed cloud services treated as concrete engineering choices rather than background magic. The Caddy-on-EC2 deployment that hosts Vakansio today is a direct descendant of the patterns he walked us through.

Dmytro Nomirovskiy ran the toughest mathematics courses of my degree. The classes were not easy, but the habit of working through a problem until the proof actually closes is something I have carried into every layer of this project, including the statistical inference that backs Chapter 5.

Finally, **Vadym Yeremenko** delivered the clearest lectures I had on programming paradigms, networking, and C++. The instinct to reach for the simpler abstraction when a problem feels tangled, and to treat the wire format as a first-class contract, came from his classes and shows up everywhere from the layer interfaces in Domain to the v6.7.8 contract drift fix described in Chapter 4.

Abstract

When I look for an IT job in Ukraine, I have to visit several job sites. Each one has its own listings, format, and search options. For every vacancy, I decide one at a time whether the role fits. The work of comparing each vacancy to my own background is repeated by every candidate, every time, and across a full search it adds up to tens of hours of reading, while I am tired and making worse choices.

In this thesis I present Vakansio, a live personalised job-matching service that helps the candidate side of the search. The system collects vacancies from seven sources, turns them and the candidate's CV into structured data, and scores every (CV, vacancy) pair. The score comes from a deterministic seven-axis sub-score calculator, refined by a Composite Large-Language-Model Judge, and the result is presented with bilingual English and Ukrainian explanations. I implemented the back-end on Clean Architecture and the SOLID principles in .NET 8, the front-end in React and TypeScript, and host the service on Amazon Web Services. It is reachable on the open internet.

I propose a five-layer evaluation methodology: CV normalisation, vacancy normalisation, score, reason text, and ranking. It follows the Holistic Evaluation of Language Models framework, with a method matched to each layer's data type. The reason-text layer reports a significant improvement of the v6 production prompt over the v4 baseline (overall $\Delta = +0.30$, 95 per cent paired-bootstrap CI $[+0.19, +0.40]$, $N \approx 391$ pairs), judged by Claude Opus 4.7. The ranking layer is reported as directionally non-significant on a gold set of 14 CVs ($\Delta \text{NDCG}@10 = -0.047$, 95 per cent paired-bootstrap CI $[-0.168, +0.065]$), judged by Claude Sonnet 4.6; a follow-up sensitivity check on the proposed calibration clamp moves the headline to $+0.018$, but the verdict stays inconclusive at the standard significance level. The methodology holds whatever the empirical verdict turns out to be, and I state openly what would be needed to settle the open question.

Keywords:

Job Matching, Large Language Models, Clean Architecture, .NET 8, Recommender Systems, LLM-as-Judge

1 | Introduction

1.1 Initial situation

When I started looking for an IT job in Ukraine, I quickly noticed how fragmented the market is. To cover it properly, I had to visit many different websites. Each one keeps its own list of vacancies, in its own format, with its own search and filter options. There is no single place where I could see everything at once, so I went through site by site, opening vacancy after vacancy.

For every vacancy I had to read the description, the list of requirements, and the company information, and then decide whether the job actually fits me. I answered the same questions each time: does the role match my background, do I have the listed skills, is the seniority right, is my English good enough, is the salary acceptable. Reading and judging a single vacancy takes a few minutes. A typical search returns dozens, sometimes hundreds of postings, so I spent many evenings on this before I even had a shortlist.

None of the popular Ukrainian IT job sites do this work for me. They show vacancies by date or in the order the site chooses, but none of them tells me, before I start reading, that one vacancy is a strong fit and another is not. I realised that this work — comparing each vacancy with my own CV — is repeated by every candidate, every time they look for a new job.

1.2 Motivation

For most people, looking for a new job is one of the more stressful and time-consuming tasks they face during their career, and I experienced this directly. It is rarely just an evening of clicking through listings. When I wanted to find a position that truly fits, I had to spend weeks on it: opening dozens of vacancies a day, comparing each one against my own background, preparing tailored applications, and keeping track of where I had already applied.

The time cost was the most visible part. When I spent three to five minutes carefully reading one vacancy and another minute or two deciding whether to apply, a single evening covered roughly twenty or thirty postings. I had to do this on top of studies and other work, so across a full search I accumulated thirty to fifty hours just on the evaluation phase, before counting interviews, technical tasks, and follow-up emails.

The hidden cost was the cognitive load. Every vacancy made me switch context — different company, different stack, different seniority, different country, sometimes a different language. Doing this for hours, day after day, leads to decision fatigue [1]. By the end of an evening, the vacancies started to blur together, and I noticed that I began to make worse decisions or skip listings I should have read more carefully. I missed some good matches; I pursued some bad ones.

The emotional cost was also significant. A long search without visible progress reduces motivation, and repeated rejection — or, more often, no reply at all — made me doubt my

own qualifications. The longer the search took, the more it affected my focus and effort, which in turn slowed the search further.

I realised that a tool that takes on part of this work — that reads each vacancy carefully, compares it against my CV, and tells me in advance where to focus — would have clear value. Saving even half of the evaluation time, and surfacing the most promising vacancies first, would turn the job search from a lengthy and exhausting process into one that is much more manageable. This is the problem I set out to solve with Vakansio.

1.3 Objectives

I built Vakansio around four explicit objectives. Each one is restated as a measurable goal in a later chapter:

1. **A working public service.** The system must be reachable on the open internet, support user registration and login, accept a CV as a PDF upload, search for vacancies from several sources, and return a personalised list with verdicts and explanations. The deployed instance is available at dsus1dizgh006.cloudfront.net and supports real user accounts.
2. **A disciplined software architecture.** The code base has to follow the Clean Architecture pattern proposed by Robert C. Martin [2] and the five SOLID principles. This is not a stylistic choice: the layer separation has to be visible in the code — the Domain project must not reference the Infrastructure project, and abstractions must live in the layer that needs them. The SOLID principles must be traceable to concrete artefacts. Section 3 documents the mapping in detail.
3. **Measurable match quality.** An informal impression is not enough. I defined an evaluation methodology that another engineer could repeat. The primary level compares the deterministic field-by-field outputs of the pipeline against my own curated reference normalisations of CVs and vacancies, using mean absolute error and verdict agreement, computed by the evaluation tool that ships with the code base. The secondary level uses an LLM as judge to rate match quality on a held-out batch of CV–vacancy pairs; ranking-quality numbers derived from these ratings are reported alongside the primary results in Section 5.
4. **Cost transparency for every LLM call.** Each call must be logged with the pipeline stage that issued it, the number of input and output tokens, and the computed dollar cost. The data is written to a PostgreSQL table and is queryable by an operator. Per-stage cost numbers and aggregate statistics appear in Section 5.

Several topics are out of scope. The service does not allow employers to post their own vacancies — vacancies are aggregated from external sources only. There is no employer-side feature for searching candidates. The deployment is single-tenant: there is no organisation account model. I did not run an external user study with multiple real candidates; the evaluation is internal, against the gold sets described in Section 5, and the implications of this choice are discussed there.

1.4 Method

I built the system as a four-layer .NET 8 back-end, a React and TypeScript front-end, and a small AWS deployment. The four layers are Domain, Application, Infrastructure, and API. The Domain layer contains entities, value objects, and pure scoring rules that do not depend on any external library. The Application layer runs the use cases — searching, scoring, and returning results — following the Command-Query Responsibility Segregation pattern. The Infrastructure layer adapts external dependencies: the Gemini large language model, the PostgreSQL database, Amazon S3 for CV files, and seven scrapers, APIs, and RSS feeds that fetch vacancies from job sites. The API layer is a thin HTTP entry point: controllers, authentication, and routing.

My scoring approach is hybrid. Some parts of candidate–vacancy fit can be computed precisely: whether the listed skills overlap, whether the seniority level matches, and how many years of experience the candidate has, among other measurable signals. These parts are computed by deterministic C# code organised as seven sub-scores: skill, seniority, experience, role intent, domain, language, and education. Their weighted sum is the first numeric estimate of fit. Other parts of the judgement are harder to express as fixed rules — for example, whether the vacancy description asks for the same kind of work the candidate has actually done. For these parts, the system calls Gemini 2.5 Flash to act as a Composite Judge that refines the deterministic score against a per-domain rubric. A second batched LLM call generates short bilingual explanations in English and Ukrainian for the top results.

The infrastructure runs on Amazon Web Services. A single t3.micro EC2 instance hosts the API container behind Caddy. The database is an Amazon RDS PostgreSQL instance. CV files live in an Amazon S3 bucket. The front-end bundle is served from a second S3 bucket through CloudFront.

I worked iteratively, around versioned releases of the scoring pipeline instead of fixed-length sprints. The methodology, coding standards, and quality-assurance approach are documented in Section 4.

This summary covers the high-level shape only; the architecture, the scoring pipeline, the technology choices, and the deployment story are documented in the chapters that follow.

1.5 Structure of this thesis

I organise the remainder of the document as follows.

Section 2 surveys the Ukrainian job-search landscape, the approaches of the main job sites, and related work in recommender systems and large-language-model evaluation. It also lists the functional and non-functional requirements that Vakansio has to satisfy.

Section 3 presents the architectural drivers, the technology stack with justification against alternatives, the four-layer Clean Architecture decomposition, the scoring pipeline and Composite Judge, the cache hierarchy, the user-experience approach, and the mapping of the SOLID principles onto concrete artefacts in the code base.

Section 4 walks layer by layer through the Domain, Application, Infrastructure, and front-end implementations, documents the deployment and operations story on AWS, and describes two engineering challenges encountered during development.

Section 5 details the evaluation methodology, the construction of the gold sets, the primary numerical results, the secondary LLM-as-judge analysis, and the threats to validity, with an end-to-end worked example in the appendix.

Section 6 revisits the four objectives stated in this introduction, recounts the difficulties encountered, and outlines future work.

2 | Research

2.1 The Ukrainian job-search landscape

Before I started designing Vakansio, I looked carefully at the Ukrainian IT market and at the way candidates search for jobs in it. The market itself is large enough to make manual job search time-consuming. At any given moment, tens of thousands of open positions are visible across the various Ukrainian job sites, with new vacancies appearing daily. Candidates looking for a position typically check several sites in parallel, because no single source covers the whole market.

The landscape is fragmented. Different sites serve different audiences. Some focus specifically on IT and product roles, some are general-purpose and cover every industry, some are international networks with a Ukrainian segment, and at least one is a meta-aggregator that re-publishes listings from other boards. Each site stores its own catalogue, requires its own account, and presents listings in its own format. A candidate cannot search across all sources from a single place — they have to repeat the same query, with the same intent, on each site separately.

Within each site, the ranking of returned results is rarely personalised to an externally uploaded CV. The default ordering is usually by date of posting, and the available filters are coarse — location, seniority, salary range, sometimes a fixed set of technology tags. Where a site does offer a personalised signal at all, it operates on the candidate's structured on-platform profile rather than on a free-form CV document, and the score is provided without an explanation of how it is derived.

These two problems — split sources and no CV-based ranking with explanation — define the gap I built Vakansio to fill. A service that aggregates vacancies from multiple Ukrainian sources and ranks them against an uploaded CV, with a human-readable explanation, is not offered by any mainstream Ukrainian platform today.

2.2 Existing job sites and what they offer

To make the gap concrete, I profiled the major sites that a Ukrainian IT candidate is likely to use. For each site I list what it does well and where it falls short for the specific problem of ranking an externally uploaded CV against multi-source listings with an explanation.

Djinni.co is an IT-focused board with a candidate-oriented design. It is the largest Ukrainian tech job marketplace, with around fifty thousand developers using the site each month. Candidates create a profile with a declared salary expectation, a technology stack, and a seniority level, and employers see anonymised candidate cards until the candidate chooses to reveal contact details.

Strengths:

- Transparent salary practices — every candidate lists net monthly salary expectations
- Strong anonymity model that protects candidates from their current employer
- Precise filters within the IT domain (language stack, salary range, English level, country, employment type)

- Free for candidates, with no hidden fees

Weaknesses:

- Default ordering of the result list is reverse-chronological
- No ranking against an externally uploaded CV document
- Coverage is limited to IT and adjacent roles
- The match signal that exists relies on the platform's own structured profile, not on the candidate's CV file

DOU.ua is one of the oldest Ukrainian developer communities, founded in 2005, with an integrated job board at **jobs.dou.ua**. Listings tend to be detailed and written by employers themselves, and the board provides a public RSS feed for automated consumption. The search interface offers keyword and category filters.

Strengths:

- Detailed listings written directly by employers
- Public RSS feed that supports automated aggregation
- Strong community presence with company reviews and salary surveys

Weaknesses:

- No personalised ranking — listings are sorted by date
- Search is limited to keyword and category filters
- Volume is lower than the general-purpose boards
- Strongest as a passive notice-board for the community rather than as a high-volume search tool

Robota.ua is a general-purpose Ukrainian job board covering every industry, with around 400,000 active job seekers and membership in the Pracuj.pl network. In 2024–2025 the platform launched an AI Job Search Agent that creates resumes, suggests personalised vacancies, and guides the candidate through a messaging-style interface.

Strengths:

- Wide coverage across industries — retail, manufacturing, white-collar, IT
- Standard filters for industry, region, schedule, and salary
- New AI Job Search Agent provides personalised suggestions and resume help

Weaknesses:

- The AI Agent operates on the platform's own structured profile, not on an externally uploaded CV document
- Suggestions are provided without a human-readable explanation of why a particular vacancy was picked
- Aggregation from external sources is not part of the offering

Work.ua is the largest Ukrainian job board by traffic, with several million unique monthly visitors and around three and a half million candidate accounts. It covers every industry and hosts the largest Ukrainian CV database by volume.

Strengths:

- Largest reach in Ukraine by visitor count
- Very large active CV database

- Broad cross-industry coverage and standard search filters
- Strong recruiter-side tools and trust algorithms introduced in 2025

Weaknesses:

- No personalised ranking based on an externally uploaded CV
- Relevance signal is keyword presence and selected filters
- No explanation accompanies the ordering of returned results
- No aggregation from competing sources

LinkedIn is an international professional network with a Ukrainian segment. Its jobs ranking uses more than three hundred signals built on top of the candidate's profile, connections, and engagement on the platform.

Strengths:

- Sophisticated ranking based on a Semantic Skill Graph
- Personalisation tied to the candidate's full profile and network
- International coverage including Ukrainian employers

Weaknesses:

- Ranking is driven by the LinkedIn profile, not by an uploaded CV document — candidates with a thin LinkedIn profile but a rich CV do not benefit
- Strict restrictions on automated access (see Section 2.3)
- No transparent explanation of why a particular vacancy was ranked highly

Jooble is a meta-aggregator founded in Ukraine in 2006. It collects listings from more than fifteen thousand public job sources worldwide and presents them in a single interface with a link out to the original posting; it is the second largest job search engine in the world by reach.

Strengths:

- Very broad coverage across many national job boards in a single interface
- Strong international reach and a Ukrainian core team
- Direct keyword search across many sources at once

Weaknesses:

- No ranking against an uploaded CV — the user sees a long list of links sorted by the same default signals as the originating sources
- Results lead out to other platforms, so the experience is not continuous
- No explanation accompanies the ordering

Several smaller and niche boards (Grc.ua, Happy Monday, Lobby.ua, Jobs.ua) operate alongside the six profiled above; they follow the same general pattern and I do not profile them separately here.

Every site I surveyed follows the same pattern: keyword search, date ordering, and coarse filters. Where a personalised signal exists at all, it operates on the site's own structured profile rather than on an externally uploaded CV, and it is provided without an accompanying explanation. None of the surveyed sites ranks an externally uploaded CV against multi-source listings with a human-readable explanation of why a particular

listing received its position. This is the specific gap that I designed Vakansio to fill, and it is restated as a concrete requirement in Section 2.7.

2.3 Legal and technical constraints on data collection

Pulling from multiple job sources raises two questions before any commercial launch: is the data collection legally defensible, and is the channel sustainable in production? I first surveyed the legal landscape in Ukraine and the European Union, then summarised the per-source position with respect to commercial use, and finally outlined the alternative service providers that a market version of Vakansio would require.

Legal landscape. The biggest risk for a Ukrainian company aggregating job listings is not criminal liability under Article 361 of the Criminal Code (unauthorised interference with information systems). Nor is it GDPR applied to vacancy text — that text, without personal identifiers, is not personal data. The main risk is civil. The 2022 Law of Ukraine on Copyright and Related Rights (No. 2811-IX), in force since 1 January 2023, introduced a *sui generis* database right that protects, for fifteen years, databases the maker has invested substantially in. Sites such as Work.ua and Robota.ua are likely to qualify, although no Ukrainian court has yet ruled on this in the context of job-board aggregation. The controlling European Union precedent for this kind of situation is the Court of Justice ruling in *CV-Online Latvia v. Melons* [3], in which the operator of a Latvian meta-search engine indexing job listings was found to “extract and re-utilise” the database of CV-Online; the Court held that such acts may be prohibited by the database maker only where they adversely affect the investment in obtaining, verifying, or presenting the database contents. The ruling is directly applicable to the situation of Vakansio.

A separate consideration applies where individual listings contain incidental personal data — for example, named recruiter contacts or sole-proprietor employer details. Any such data I ingest through the prototype’s scraping channels is processed under the same legitimate-interest basis or stripped on ingestion. The collection of CV files uploaded by the candidate is governed by a different basis. CV files contain personal data and trigger the full set of obligations under the Ukrainian Law on the Protection of Personal Data and, where the candidate is in the European Union, the General Data Protection Regulation. These obligations include an explicit lawful basis for the processing, retention limits, and the candidate’s right to access and deletion. I honour them in Vakansio through explicit consent at upload time, scope limitation (CVs are used solely for matching), and deletion on account closure.

Recent litigation against scrapers of large platforms provides relevant context. In December 2022 the *hiQ Labs v. LinkedIn* proceedings ended with a stipulated judgment under which hiQ paid USD 500,000 and accepted a permanent injunction. This outcome stood despite an earlier appellate ruling that scraping publicly accessible data is not a violation of the United States Computer Fraud and Abuse Act. In July 2025 the LinkedIn data provider Proxycurl shut down following a permanent injunction in *LinkedIn v. Proxycurl*. These outcomes indicate that, while scraping public data is generally not a criminal matter, civil liability arising from breach of platform terms of service is real and has materialised against well-funded specialised providers.

Per-source status. The seven sources I currently use in the Vakansio prototype have different statuses with respect to commercial use. Table 1 summarises each one, together with the channel that a commercial version would have to adopt.

Source	Status for commercial use	Channel for product
DOU.ua	Permitted via official RSS feed	Keep current RSS channel
Jooble	Permitted via official REST API	Keep current API channel
Djinni.co	Negotiable through partnership	Partner API via direct outreach
Robota.ua	Restricted; aggregator access dis-allowed	Business-development partner-ship
Work.ua	Restricted by terms of service	Partnership through partner con-tacts
LinkedIn	Prohibited; high legal risk	Paid licensed data provider
Manual URL	User-driven, no policy issue	Keep as is

Table 1: Per-source status for commercial use and the channel a market version would adopt.

Paid replacement for LinkedIn. Direct scraping of LinkedIn is not a sustainable commercial channel; replacement is therefore required. Several paid providers offer LinkedIn job-listing data on more clearly licensed terms. Apify, Bright Data, and JSearch make the data available through scraping-as-a-service offerings at approximately USD 100–300 per month for the volume needed by a small commercial deployment; these providers outsource the technical collection but do not assume contractual liability for breaches of LinkedIn terms of service. Coresignal positions itself as a licensed data vendor and provides equivalent coverage at approximately USD 1,000 per month under a negotiated commercial agreement, which is the option offering the most clearly delineated legal basis for a commercial launch. The LinkedIn Talent Solutions partner programme is, in practice, not accessible to a pre-revenue startup.

Path to commercial deployment. The mix I propose for a market version of Vakansio is therefore as follows. The free legitimate channels — the DOU RSS feed and the Jooble API — remain unchanged. A new channel is added through the OLX Partner API, which the prototype does not currently use. A parallel applicant-tracking-system aggregation channel is built on top of the free public boards exposed by Greenhouse, Lever, Workable, and SmartRecruiters; this channel covers Ukrainian-relevant employers such as Grammarly, GitLab, MacPaw, Kyivstar, and Ajax Systems, and brings several thousand additional openings without per-listing cost. Direct scraping of Djinni, Robota.ua, and Work.ua is replaced by partnership-based access through the data-licensing routes described above. Direct scraping of LinkedIn is replaced by a licensed paid provider — Coresignal for a production launch on a clearly delineated legal basis, or Apify or Bright Data for a minimum-viable product.

The Vakansio implementation I evaluate in this thesis is a capstone prototype. The scrap-ing channels currently in use are appropriate for that purpose: they access only publicly

visible listings, do not bypass authentication, respect robots.txt where it expresses an explicit position with respect to the public listing pages, and operate at human-level rates. Commercial deployment would replace those channels with the official, partnership, and paid options described above.

2.4 Related work in recommender systems

A recommender system is software that suggests items to a user — products in an online shop, films on a streaming service, or vacancies on a job board. Recommender systems have been studied since the mid-1990s, and Gediminas Adomavicius and Alexander Tuzhilin [4] distinguish three main families of approach.

The first family is **content-based**. The system looks at facts the user has shared about themselves — for Vakansio, the uploaded CV — and finds items whose own description matches those facts. If the CV says the candidate knows React and TypeScript and has three years of experience, a content-based recommender will rank highly the vacancies that ask for React, TypeScript, and three or more years of experience.

The second family is **collaborative filtering**. The system looks at what other users have done — which items they liked, viewed, or applied to — and recommends the items that similar users picked. For Vakansio, this would mean ranking a vacancy highly because other candidates with a profile similar to the current candidate’s applied to it.

The third family is **hybrid**. The system combines content-based matching and collaborative filtering and weighs the two signals together.

I chose the content-based family for Vakansio. There are three reasons. First, a candidate who signs up for the first time has no past behaviour for the system to learn from, which means a collaborative filter would have nothing to compare against. Second, the CV and the vacancy both contain structured fields. This makes attribute matching simple. Third, the candidate expects an explanation of why a vacancy was ranked highly, and content-based matching can give that explanation directly, because the match is described using attributes from the candidate’s CV.

A separate question is how to measure whether a ranked list is good. A standard answer in information retrieval is a metric called **normalised discounted cumulative gain** (NDCG). The idea is simple: items shown higher in the list have more impact on the user, so the metric gives more weight to relevance at the top of the list and less to relevance at the bottom. The variant NDCG@10 measures the quality of the first ten results. It considers both how relevant each result is and how high it appears in the list. NDCG was first defined by Kalervo Järvelin and Jaana Kekäläinen [5]. In this thesis I compute NDCG@10 in a separate offline analysis presented in Section 5; it is not part of the production scoring pipeline.

2.5 LLM-based evaluation frameworks

Evaluating the output of a large language model is harder than evaluating a classifier. The output is free-form text rather than a single label, and several different answers can all be reasonable. Two recent lines of work address this difficulty and are relevant to my evaluation methodology.

The first is HELM — Holistic Evaluation of Language Models — published by Percy Liang et al. [6] at Stanford. The main idea of HELM is to evaluate a language model on many separate metrics rather than a single score. Instead of asking whether the model is good in general, HELM asks how accurate it is on a given task, how robust it is to small input changes, how calibrated its confidence scores are, and how fair it is across different groups of inputs, among other dimensions. Each metric is computed separately, so that strengths and weaknesses become visible. I borrow this multi-metric idea: every stage of the pipeline — CV normalisation, vacancy normalisation, sub-score correctness, ranking quality, and reason quality — is evaluated on its own metric, instead of a single accuracy number that hides where the system fails. The full layered methodology is described in Section 5.

The second is LLM-as-Judge — using a strong language model as the evaluator for the output of another model — proposed by Lianmin Zheng et al. [7] in the context of MT-Bench and Chatbot Arena. The argument is empirical: when human annotators and a strong language-model judge (GPT-4 in the original paper) are asked to compare the same pair of answers, they agree more than 80% of the time. This makes the language-model judge a cheaper substitute for human ratings, especially when the evaluation set is too large to be rated by hand. I use the LLM-as-Judge pattern in two places in Vakansio. Inside the scoring pipeline, a Composite Judge step refines the deterministic scores against a per-domain rubric. Inside the evaluation, a separate large language model is asked to rate a held-out set of CV–vacancy pairs on an ordinal scale; those ratings serve as the ground truth against which I compare the production ranking in Section 5.

2.6 Gaps and my positioning

Sections Section 2.1 to Section 2.5 surveyed two bodies of work that are relevant to Vakansio: the existing Ukrainian job-search market on the product side, and the academic literature on recommender systems and language-model evaluation on the methodology side. I now pull the two together and state explicitly what Vakansio is and how it relates to each.

The product-side gap is specific. None of the Ukrainian job sites offers a personalised ranking of multi-source vacancies against an externally uploaded CV, accompanied by a human-readable explanation of why each match was made. This is true for the IT-focused boards such as Djinni and DOU, for the general-purpose boards such as Robota.ua and Work.ua, and for the international platforms such as LinkedIn and Jooble. Where partial elements exist — for example Robota.ua’s AI Job Search Agent over its on-platform profile, or LinkedIn’s ranking based on the LinkedIn profile and engagement signals — they are tied to a single source’s internal representation of the candidate, and they are not accompanied by an explanation. I built Vakansio to fill this combined gap.

On the methodology side, I apply a specific combination of ideas from recent work. Vakansio is a content-based recommender (Section 2.4), which is the natural choice for a system that has to deliver useful recommendations from the candidate’s very first session and that has to explain each recommendation in terms of the candidate’s own CV. The scoring pipeline computes seven deterministic sub-scores and refines them through a Composite Judge step that follows the LLM-as-Judge pattern (Section 2.5). The evaluation methodology described in Section 5 follows HELM’s multi-metric approach (Section 2.5)

— each stage of the pipeline has its own metric — and uses an LLM-as-Judge to construct the held-out ratings against which I measure ranking quality.

My contribution is not in any single one of these elements. Content-based recommendation is well established, the LLM-as-Judge pattern has been published, and multi-metric evaluation is the standard direction of the field. My contribution is the combination: a multi-source CV-driven matching service for the Ukrainian market, delivered with explanations, built on a hybrid scoring pipeline that combines deterministic sub-scores with an LLM judge, and evaluated with a layered methodology that reports each stage’s quality separately instead of rolling everything up into one number.

2.7 Functional and non-functional requirements

The work in Sections Section 2.1 to Section 2.6 motivates a specific set of requirements that Vakansio must satisfy. I group the requirements into two families. The first family captures what the system must do, expressed as user-testable behaviour. The second family captures how well the system must do it, as properties that matter once the system is deployed.

Functional requirements. The system must:

1. **FR1 — Account management.** Allow a candidate to register an account and log in.
2. **FR2 — CV upload and parsing.** Accept a CV as a PDF upload and parse it into a structured representation.
3. **FR3 — Multi-source search.** Search for vacancies from several external sources in response to a keyword query.
4. **FR4 — Personalised ranking.** Return a ranked list of vacancies, ordered by the system’s estimate of fit between the uploaded CV and each vacancy.
5. **FR5 — Verdict and explanation.** Show, for each returned vacancy, a numerical match score between zero and one, a verdict label (Strong, Partial, Weak, or Mismatch), the matched skills, the missing must-have skills, and a short bilingual explanation in English and Ukrainian.
6. **FR6 — Filtering.** Support filtering of returned results by source, seniority level, and work format.
7. **FR7 — Application tracker.** Allow a candidate to save vacancies of interest to a personal application tracker and to update their application status.

Non-functional requirements. The system must satisfy the following properties:

1. **NFR1 — Public deployment.** The system is reachable on the open Internet at a stable URL, s

3 | Design

3.1 Architectural drivers

In this chapter I describe the design of the Vakansio system. Every major decision traces back to one or more requirements from Section 2.7. The strongest drivers — the requirements that shape several design decisions at once — are listed below as a roadmap to the rest of the chapter.

The first driver is the combination of **multi-source aggregation**, **CV-driven ranking**, and **human-readable explanation**. It calls for pluggable source adapters so that each external job board can be added or replaced in isolation, a single centralised scoring pipeline through which every (CV, vacancy) pair passes, and a structured output that carries the score, the verdict, the matched skills, the missing must-haves, and the bilingual reason text. The responses appear in the technology stack section (Section 3.2), the Clean Architecture layering (Section 3.3), and the scoring pipeline design (Section 3.4).

The second driver is **production deployment at predictable cost**. The system has to stay reachable on the open Internet without manual intervention. This drives containerisation of the back-end, the use of managed services for the database and object storage, a reverse proxy that handles TLS automatically, and a static-hosting model for the front-end. The deployment topology is documented in Section 3.7.

The third driver is **measurable layer separation**. SOLID compliance has to be visible in the code, not only claimed. This requires a strict four-project decomposition (Domain, Application, Infrastructure, API), interfaces declared in the layer that uses them, and a command/query split that keeps each handler single-purpose. The decomposition is covered in Section 3.3.

The fourth driver is **per-stage evaluability and cost transparency**. Each pipeline stage must produce a measurement and a cost record. This drives a modular pipeline whose stages can be evaluated independently, a request-scoped accumulator that gathers per-stage cost without coupling the layers, a persistent cost-ledger table for offline querying, and an offline evaluation harness separate from the production code path. The responses appear in the scoring pipeline design (Section 3.4) and the observability section (Section 3.8).

The fifth driver is **security and data protection**. Uploaded CV files and user accounts are subject to the Ukrainian Law on the Protection of Personal Data and, where applicable, the General Data Protection Regulation. The architectural responses are stateless authentication based on JSON Web Tokens, secret storage in the AWS Systems Manager Parameter Store rather than the source tree, startup guards that refuse to start the system without an enforced SSL connection string and a CORS allow-list, and a cascade-delete pathway for full account removal. These are described in Section 3.8.

The sections that follow turn each driver into a concrete architectural choice.

3.2 Technology stack selection

This section presents the major technology choices behind Vakansio. For each choice I state what I picked, the alternatives I considered, and the reasons I picked it over them, with reference to the requirements stated in Section 2.7.

Back-end runtime: .NET 8. I implemented the back-end in C# on the .NET 8 runtime, over Node.js with TypeScript, Go, and Python with FastAPI.

- **Architecture fit:** ASP.NET Core has a mature dependency-injection container, a configuration-binding system, and a middleware pipeline that align well with the Clean Architecture and Options patterns required by Section 2.7.
- **Type safety:** C# offers a strong static type system with nullable reference types, which makes the interface contracts at layer boundaries enforceable at compile time.
- **Practical experience:** I already had working knowledge of the .NET ecosystem, which kept the development pace realistic for a solo project.

Front-end framework: React and TypeScript on Vite. I built the front-end as a single-page application in React with TypeScript, bundled by Vite, over Vue 3 and a server-side rendered framework such as Next.js.

- **Ecosystem:** React has the largest npm download share among SPA frameworks and a deep set of companion libraries — Zustand for client-side state, TanStack Query for server data fetching, React Router for routing — each of which Vakansio uses.
- **Type safety:** TypeScript gives the same compile-time contract enforcement on the front-end that C# gives on the back-end.
- **Build speed:** Vite provides fast hot-module reloading during development and a clean production build with no `webpack.config.js` to maintain.
- **Why not SSR:** The user interface is stateful and client-driven; server-side rendering would add operational cost without a search-engine-optimisation benefit on an authenticated application.

Database: PostgreSQL via Amazon RDS. I chose relational storage over a document store, picking PostgreSQL over MySQL and MongoDB.

- **JSON fit:** First-class `jsonb` columns let normalised vacancy and CV representations live alongside relational data without a separate document database.
- **SQL features:** Window functions, full-text search, and advisory locks cover the analytical queries that the cost-ledger and evaluation paths require.
- **Managed offering:** Amazon RDS removes the operational overhead of running and backing up the database, which matters for a solo project.
- **Why not MySQL:** It lacks native `jsonb` support, which the normalisation cache relies on.
- **Why not MongoDB:** The data model does not require schema-less storage, and PostgreSQL’s strong-consistency guarantees simplify the cache-invalidation reasoning described in Section 3.5.

Large language model: Gemini 2.5 Flash. I chose Gemini 2.5 Flash for the CV normalisation, vacancy normalisation, Composite Judge, and reason-generation stages,

over OpenAI’s GPT-4-class models, Anthropic’s Claude, the Groq inference service, and a locally hosted open-weight model.

- **Cost per token:** Materially lower than GPT-4 and Claude at the scale of the pipeline.
- **Structured output:** Native JSON output simplifies parsing in the scoring pipeline.
- **Latency:** Acceptable at the volumes seen by Vakansio.
- **Rubric following:** The Composite Judge benefits from a model that can follow a long calibration rubric reliably and return scores within a strict numerical range.
- **Why not local:** The t3.micro deployment target does not have enough memory or compute to serve a model with comparable instruction-following quality.

Cloud platform: Amazon Web Services. I hosted the system on AWS, over Google Cloud Platform, Microsoft Azure, and DigitalOcean.

- **Familiarity:** My prior working experience with the AWS console and command-line tooling.
- **Free tier:** The AWS twelve-month free-tier programme covers one t3.micro Elastic Compute Cloud instance, twenty gigabytes of Relational Database Service storage, and modest Simple Storage Service traffic — together this covers the full production cost for a solo capstone project.
- **Edge network:** CloudFront ensures low front-end latency for users across Ukraine without operating a separate content-delivery network.

Supporting libraries. Three further choices are worth noting. I chose **Entity Framework Core** over Dapper for its first-class migration tooling and its tight integration with the .NET DI container. I chose **MediatR** over a hand-rolled dispatcher for the ecosystem of pipeline behaviours (validation, logging, retry) available through its abstraction. I chose **BCrypt.Net-Next** for password hashing because it implements the standard work-factor parameter and has a long track record of independent audits.

3.3 Clean Architecture layering and SOLID mapping

This section addresses the third architectural driver from Section 3.1. The requirements in Section 2.7 explicitly demand that SOLID compliance be visible in the code and that the layer separation be measurable rather than merely claimed. Clean Architecture [2] provides the structural answer, and SOLID provides the class-level answer; in this section I map both onto the Vakansio code base.

The four layers. I organised the back-end as four .NET projects, named for the role they play.

- **Domain** contains entities, value objects, enumerations, and pure scoring rules that describe the business of candidate–vacancy matching. It depends on no other project and on no third-party library — **Domain/Domain.csproj** declares zero package references.
- **Application** contains use cases (expressed as MediatR handlers), options classes that configure them, and abstractions through which the use cases call out to the rest of the system. It depends only on Domain.
- **Infrastructure** provides the concrete implementations of those abstractions — the Entity Framework Core data context, the Gemini service clients, the AWS S3 adapter,

the SSM Parameter Store reader, and the seven job-source adapters — and depends on Application (with Domain pulled in transitively).

- **API** is the thin HTTP entry layer: ASP.NET Core controllers, JWT middleware, the CORS configuration, options binding, and the startup file **Program.cs**. It depends on Application and Infrastructure.

The Dependency Rule. The decisive property of Clean Architecture is that source-code dependencies point inward only. I enforce this through the composition of project references: Domain is reference-free, Application references only Domain, Infrastructure references Application (and Domain transitively), and API references Application and Infrastructure. Nothing can make the Domain layer depend on, say, the Gemini SDK or Entity Framework Core. The compiler enforces the rule at build time. The same property makes the Domain layer trivially unit-testable, because there is no infrastructure to mock out.

Dependency Inversion in practice. The Application layer expresses what it needs through interfaces declared in its own folder (**Application/Common/Interfaces/**): **ICvFileStorage**, **ICostLogService**, **IScoringService**, **IBatchedJudgeService**, **IBatchedReasonService**, and others. Repository contracts owned by the entities themselves (for example **IJobVacancyRepository**) live in Domain instead. The Infrastructure layer provides the implementations. The composition root in **Program.cs** wires implementations to interfaces through the built-in .NET dependency-injection container. Handler classes in the Application layer never see a concrete class directly — every collaborator is injected through an interface. This makes the dependency direction explicit at every call site and allows entire infrastructure components to be substituted in tests or replaced in production (for example, swapping Gemini for OpenAI would touch only the Infrastructure layer).

CQRS via MediatR. Use cases in the Application layer follow the Command–Query Responsibility Segregation pattern, with separate handler classes for read-only operations (under **Application/*/Queries/**) and state-changing operations (under **Application/*/Commands/**). Each handler implements **IRequestHandler<TRequest, TResponse>** and exposes exactly one **Handle** method. MediatR dispatches incoming requests to the right handler and supports cross-cutting concerns through its pipeline-behaviour abstraction. The pattern keeps individual handlers single-purpose: the largest handler, **GetAggregatedJobsV6Handler**, orchestrates the full search-and-score pipeline but contains no Gemini calls, no Entity Framework Core context, and no scraping logic; it composes those concerns through the interfaces injected into its constructor.

Versioning as cache-invalidation. A small but architecturally significant idiom is the use of version strings to drive cache invalidation. The Composite Judge prompt body carries a **BodyVersion** constant; the scoring pipeline exposes a composite **VersionWithJudge** string that concatenates the scoring version, the judge body version, and the model identifier; the persisted **ScoringCache** row stores the composite version it was generated against. When any underlying constant changes (because a prompt was edited or a weight was rebalanced), the resulting cache key changes, and every previously cached row

becomes unreachable by readers without any explicit invalidation pass. Cache correctness therefore does not depend on a separate operational procedure.

SOLID mapping. I map the five SOLID principles onto concrete artefacts in the Vakansio code base as shown in Table 2.

Principle	Concrete artefact in Vakansio
Single Responsibility	Domain/Scoring/Verdict.cs and Domain/Scoring/GenericGapFilter.cs each contain a single rule and no orchestration; GetAggregatedJobsV6Handler delegates all stages to injected services rather than executing them inline.
Open/Closed	Application/Common/Configuration/ScoringOptions.cs together with the IOptions<T> pattern makes tuning knobs such as SyncNormalizeTimeoutSeconds configurable through appsettings.json without recompilation.
Liskov Substitution	The scoring services depend on the interfaces IScoringService , IBatchedJudgeService , and IBatchedReasonService alternative implementations against another LLM provider would satisfy the same contracts with no consumer changes.
Interface Segregation	The three scoring interfaces are kept as separate single-purpose contracts rather than fused into one large interface; a class that implements IBatchedReasonService is not forced to also provide judging or scoring methods.
Dependency Inversion	Application handlers take only abstractions in their constructors; the composition root in Program.cs wires concrete implementations to those abstractions at startup, so the Application layer never imports an Infrastructure type directly.

Table 2: SOLID principles mapped onto concrete artefacts in the Vakansio code base.

Each row points to a file path that can be opened and read.

3.4 Scoring pipeline design

In this section I present the design of the scoring pipeline — the part of Vakansio that converts a (CV, vacancy) pair into a numerical score, a verdict label, and a bilingual explanation. The pipeline is the system’s core technical contribution and the artefact that the validation in Section 5 evaluates.

Pipeline overview. Once the CV and the vacancies returned for the candidate’s query have been normalised, every (CV, vacancy) pair flows through the same sequence of stages: deterministic sub-scoring, an anti-flag penalty, an optional Composite Judge refinement, a safety cap, and finally a batched reason-generation step that produces the bilingual explanation. The shape of the pipeline is fixed; the stages are wired together by the scoring service implemented in **ScoringServiceV2.cs**.

The seven sub-score axes. The deterministic part of the score is a weighted sum of seven axis-specific sub-scores, each computed by a pure C# calculator under **Infrastructure/RelevancePipeline/V2/Scoring/SubScoreCalculators/**. The axes and their weights, taken from **ScoringServiceV2.cs** (lines 138–147), are shown in Table 3. The weights are not the result of an automated search — they are explicit design choices I made. The most recent rebalance (Skill 0.30 → 0.40, Language 0.10 → 0.05, Education 0.05 → 0.02) came from one observation: Language and Education scores are uniformly high across the candidate pool, so they add little to discrimination. The Skill score carries most of the signal that separates a good match from a poor one.

Axis	Weight	Note
Skill match	0.40	Primary discriminator (rebalanced from 0.30)
Role-intent match	0.15	
Seniority match	0.15	
Experience match	0.15	
Domain alignment	0.08	Modest reduction from 0.10
Language match	0.05	Was 0.10 — near-uniform high
Education match	0.02	Was 0.05 — near-uniform high
Total	1.00	

Table 3: Composite weights of the seven scoring axes, as configured in **ScoringServiceV2.cs:138–147**.

The deterministic linear score is the weighted sum

$$S_{\text{linear}} = \sum_{i=1}^7 w_i \cdot s_i, \quad \text{where } \sum_{i=1}^7 w_i = 1.00.$$

Anti-flag penalty. The deterministic linear score is then multiplied by an anti-flag penalty between zero and one. The penalty is produced by **AntiFlagEvaluator.cs** and triggers on conditions that suggest a candidate is structurally unsuited to the vacancy (for example, the vacancy requires on-site work in a city the candidate is unwilling to relocate to). I made the penalty multiplicative rather than additive so that a single strong anti-flag can pull an otherwise strong score down decisively.

Role-family routing. Not every vacancy belongs to the same job family, and the relative weight of “what counts as a strong skill match” differs across families. Before the Composite Judge step, each vacancy is routed to one of a fixed set of role families — Product, Engineering, Data, Design, or a Generic fallback — by **KeywordRoleRouter.cs**. The router scores each candidate family by counting weighted matches of family-specific keywords in the vacancy title (with weight five) and description (with weight one); the family with the highest score above a confidence threshold of 0.30 wins, otherwise the routing falls through to Generic. The selected family in turn selects the Composite Judge calibration examples used in the next step.

Composite Judge with skip-band. The Composite Judge is a Gemini 2.5 Flash call that takes the (CV, vacancy, deterministic score, selected sub-scores) tuple and returns a refined score together with a brief justification. The judge prompt, defined in `JudgePromptCore.cs`, includes a calibration rubric that anchors specific score bands to qualitative descriptions (0.85–0.95 EXCELLENT, down to 0.20–0.25 MISMATCH) and a set of family-specific worked examples. The Composite Judge is gated by configuration (`ML:EnableCompositeJudge`) and includes a skip-band optimisation. Pairs whose deterministic linear score falls outside the range $[0.30, 0.85]$ and have no active anti-flags skip the Judge entirely: the deterministic signal is already extreme, so a Judge call would only confirm it and spend a model invocation. I observed that the skip-band reduces the number of Judge calls per query by approximately thirty to forty per cent.

Sub-score caps as a safety layer. After the Composite Judge step, the candidate score passes through `IScoringCapService`, which applies a floor of 0.20, a ceiling of 0.88, and a set of asymmetric caps that limit how high a score can go when a candidate has a clear seniority gap, a clear language gap, or a clear role-family mismatch. The caps act as a belt-and-braces safety layer: even if the language-model judge returns a misleadingly high score for an unsuitable match, the cap pulls it back into a range that reflects the structural mismatch.

Two-pass ranking. The full pipeline above is expensive to run on every candidate vacancy because the Composite Judge and the reason generation are model calls. To keep the per-request cost manageable, I run a cheap pre-filter first. The deterministic linear score is computed for all returned vacancies, and only the top forty (configurable through `PreFilterTopN`) are passed on to the Composite Judge stage. Of those, the top thirty (configurable through `ReasonGenerationCap`) are passed to the batched reason-generation step. The two-pass shape is a deliberate cost–quality trade-off: the cheap pre-filter handles the easy decisions where the deterministic score is decisive, and the model calls are reserved for the cases where refinement is most likely to change the ranking.

Reason generation. For the top thirty pairs, a batched reason-generation step produces a structured JSON response that contains six text fields per pair — strengths, gaps, and recommendation, each in English and Ukrainian. The work is split into chunks of ten pairs per Gemini call, with up to three chunks dispatched in parallel. Each call uses `temperature=0.1` for predictability, validates the response against a strict schema, retries once if the schema check fails, and finally falls back to a deterministic template if both attempts fail. The bilingual JSON schema lets the front-end render the appropriate language without a second round trip.

Skill expansion and canonicalisation. Before any sub-score is computed, the raw skill tokens from both the CV and the vacancy are expanded and canonicalised against a global vocabulary. `SkillCanonicalizer.cs` maps synonyms (for example “React.js”, “ReactJS”, and “react”) onto a single canonical form, and the global-vocabulary path in `SkillVocabularyService` handles rare or compound terms that the static vocabulary does not cover, through a single batched Gemini call per request. I introduced the global-vocabulary path in version 6.3 to replace an earlier per-entity expander that produced many false-negative skill matches due to surface-form variation.

Evidence pipeline. The matched and missing skill chips that the front-end shows for each vacancy are not the raw output of the sub-score calculators. They pass through a four-step evidence pipeline — Build, Sanitise (deduplicate, strip CEFR tokens, drop self-contradictions), Prioritise (Tier 1 brand acronyms first, Tier 3 generic concepts last), and Trim (at most twelve matched skills and eight missing must-haves). This pipeline is what makes the Tier-based progressive disclosure on the front-end (Section 3.6) feasible.

3.5 Cache hierarchy

The scoring pipeline is expensive: each cold (CV, vacancy) pair requires several model calls. Caching is therefore not an optimisation but a structural requirement. I organised Vakansio around a four-layer back-end cache hierarchy, supplemented by a client-side cache on the front-end. Each layer answers a different question about cost and freshness.

L1 — Response cache. In-process **IMemoryCache** with a five-minute TTL, keyed by user identifier, CV version, and search parameters. Repeated identical searches by the same user are served from memory without rerunning the pipeline.

L2 — Scoring cache. PostgreSQL **ScoringCache** table indexed by the composite key (**CvHash**, **VacancyId**, **ScoringVersion**). The CV hash is the SHA-256 of a canonical CV projection, so a non-substantive edit (whitespace or field-order changes) does not change the key. There is no time-based expiry — invalidation is by version-string bump in the composite key (Section 3.3).

L3 — Vacancy normalisation. The structured vacancy representation produced by the vacancy-normalisation Gemini call is stored as a **jsonb** column (**VacancyAnalysisJson**) on the **JobVacancy** row. This is the most expensive single normalisation step in the pipeline. Once computed, it remains valid until the vacancy is re-extracted.

L4 — Scrape cache. In-process **IMemoryCache** with a thirty-minute TTL, keyed by lower-cased keywords and location. This is the layer that lets a second identical search reuse the previous scrape rather than pull every source from the network again.

Front-end cache. TanStack Query persisted to IndexedDB. The persisted cache has a seventy-two-hour maximum age and a ten-minute stale time; writes are throttled to one per second; a cache-buster version string ensures that incompatible deployments do not serve stale data; and a multi-tab synchronisation channel keeps several open browser tabs consistent with one another.

The four back-end layers use different keying strategies but share a single invalidation philosophy. Where time-based expiry is appropriate (L1 response and L4 scrape) it is short, because the underlying data is volatile. Where time-based expiry is inappropriate (L2 scoring and L3 normalisation) the key itself encodes the version of the procedure that produced the cached value, so a procedure change drops the entire affected slice without any explicit operation. This pattern is what makes the scoring pipeline safe to evolve without coordinated cache flushes.

3.6 Frontend architecture and UX design

The front-end is a single-page React application written in TypeScript and bundled by Vite. It serves five distinct pages — Login, Register, Profile, Job Feed, and Tracker — and

acts as the visible surface of the system. In this section I describe the organisational and user-experience design decisions on the front-end side.

Component organisation. I separated three concerns in the source tree.

- **components/ui/** holds design-system primitives — Badge, Button, Card, Input, Modal, Icon — that carry no business-domain knowledge.
- **components/jobs/** holds feature-level components specific to vacancy display — VerdictBadge, EvidenceChips, VacancyDetailDrawer, and the JobCard variants.
- **pages/** holds the route-level components that React Router maps to URLs.

Cross-component contracts that would otherwise drift between files are pinned to single-source-of-truth maps: for example, **verdictMeta.ts** defines the verdict label and colour once, and every place that renders a verdict reads from this map rather than maintaining its own table.

State management. Two separate state systems coexist on the front-end, each with a clearly defined role.

- **Zustand** stores cross-cutting UI and session state (the search term, the active filter selection, the authentication token).
- **TanStack Query** stores server data (the result of an API call together with its loading and error state).
- **React component-level state** stores strictly local data — for example, the identifier of the currently open detail drawer — because it has no readers outside its owning component.

The split keeps the React component tree free of imperative state-management code and avoids the common anti-pattern of caching server data in component-level local state.

Job-feed and detail-drawer pattern. The core screen of the application is the Job Feed. Vacancies are displayed as cards in a scrollable list; clicking a card opens a side drawer containing the full vacancy description and the structured explanation. I made the drawer a sibling component of the list rather than a separate route deliberately: a route change would invalidate the in-page list state (scroll position, filter selections, expanded chip groups) and force a re-render of the result list. The drawer pattern keeps the underlying list mounted and reuses the already-fetched data for the detail view.

Tier-based progressive disclosure of evidence. Each vacancy carries up to twelve matched-skill chips and eight missing-must-have chips. To prevent the user from being overwhelmed by a wall of chips, and to make scanning useful when the chip count is high, I classify the evidence chips by **evidenceTier.ts** into three tiers based on structural shape.

- **Tier 1** — brand names and tool acronyms (Figma, Mixpanel, GA4), shown by default.
- **Tier 2** — hyphenated or multi-word skills, shown with a slight visual hierarchy.
- **Tier 3** — generic concepts (“communication”, “user-centric approach”), hidden behind a single “+N similar” expansion link.

The classifier carries exception lists for known false positives — **LOWERCASE_BRAND_TOOLS** rescues lowercase brand names such as “react” or “spring” from being demoted,

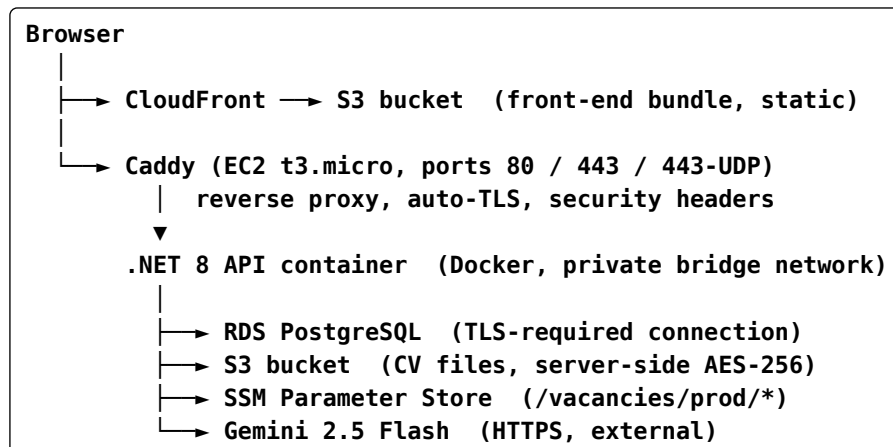
and **IMPLICIT_PM_METRICS** rescues product-manager metric phrases — so the classifier behaves correctly on shapes that the heuristic alone would misclassify. The tier policy mirrors the back-end evidence-pipeline priorities described in Section 3.4.

Verdict badges and bilingual UX. Each vacancy is labelled with a coloured verdict badge — **Strong** (green), **Partial** (blue), **Weak** (amber), or **Mismatch** (red) — read from **verdictMeta.ts**. The reason text is bilingual: the back-end returns both English and Ukrainian strings, and the front-end renders whichever language the user has selected through the **LanguageContext**. The internationalisation layer uses paired locale tables (**en.ts** and **uk.ts**), a **useT** hook for translated strings, and a **usePlural** hook for plural-form handling, so no user-visible string is hard-coded.

Persistence subsystem. A dedicated module under **frontend/src/persistence/** owns the lifecycle of the front-end cache. It provides a migration step that upgrades the schema between application versions, a multi-tab synchronisation channel that propagates writes across browser tabs, a quota check that prevents the IndexedDB cache from exceeding browser-imposed limits, and an eviction policy that drops the persisted cache blob when it exceeds a fifteen-megabyte hard cap. The subsystem is what makes the TanStack Query persisted cache (described in Section 3.5) safe to leave running across multiple sessions.

3.7 Deployment topology

I designed the deployment topology of Vakansio around three goals: keep the public surface small and standard, lean on managed services wherever the operational gain outweighs the configuration cost, and stay within a single-author project’s operational budget. The result is a single-instance back-end, three managed AWS components, and an edge cache for the front-end. The overall shape is shown in Listing 1.



Listing 1: Vakansio production topology on Amazon Web Services.

Compute host. The application back-end runs on a single Amazon Elastic Compute Cloud t3.micro instance in the eu-central-1 region. The instance hosts two Docker containers managed by a single Compose file: Caddy at the public edge and the .NET 8 API container behind it on a private bridge network. There is no separate orchestrator — no Kubernetes, no Elastic Container Service. For a single-author project with one production environment, the operational complexity of an orchestrator outweighs the benefit.

Reverse proxy and TLS. Caddy serves as the public entry point on ports 80, 443, and 443 UDP (the last for HTTP/3). It terminates TLS using automatically issued and renewed Let's Encrypt certificates, proxies application traffic to the API container, and applies a layer of security headers (HSTS, X-Content-Type-Options, X-Frame-Options DENY, Referrer-Policy). I set a 600-second back-end timeout in the Caddy configuration to accommodate cold Gemini calls in the scoring pipeline, which can exceed the default proxy budget on the first request after a cache miss.

Database. PostgreSQL runs on the Amazon Relational Database Service in the same region as the application. The connection is enforced over TLS — the API refuses to start in production unless the connection string carries **SslMode=Require, VerifyCA, or VerifyFull**. The database is reachable only through a security-group rule that admits the application instance.

Object storage. Uploaded CV files are stored in an Amazon Simple Storage Service bucket with server-side encryption (AES-256), under a per-user key prefix of the form **users/{id}/cv/{timestamp}-{filename}**. The API exposes CV files to the front-end through pre-signed URLs valid for five minutes, rather than streaming the bytes through the API itself.

Secret management. Production configuration secrets — the database connection string, the Gemini API key, the JWT signing key, and others — live in the AWS Systems Manager Parameter Store under the prefix **/vacancies/prod/***. The API loads them at startup through the AWS SDK's **AddSystemsManager** extension method and refreshes them in the background every five minutes, so secret rotation does not require a container restart.

Front-end delivery. The compiled front-end bundle is served from a separate Simple Storage Service bucket behind CloudFront. The CloudFront distribution provides geographic edge caching for users across Ukraine, which keeps the time-to-first-byte small even though the application back-end is hosted in Frankfurt.

Identity and access control. The Elastic Compute Cloud instance carries an Identity and Access Management instance profile that grants the application read access to its parameter-store namespace and read-write access to the CV bucket. There are no long-lived AWS access keys in the source tree or in the container's environment.

Single-instance trade-offs. I deliberately picked one back-end instance despite the obvious limits — a t3.micro is the smallest viable size, and there is no horizontal redundancy. The choice keeps the operational cost within the AWS twelve-month free tier, eliminates a class of distributed-systems concerns (session affinity, distributed cache coherence) that would consume engineering time without producing thesis-relevant evidence, and is consistent with the scope of a capstone project. The migration path to a multi-instance topology would replace the Compose file with an Elastic Container Service task definition and place an Application Load Balancer in front of the instances; the layer separation described in Section 3.3 makes this migration largely mechanical.

3.8 Observability and security design

The fourth and fifth architectural drivers from Section 3.1 — per-stage evaluability with cost transparency, and security with data protection — share a common pattern. Both

rely on cross-cutting infrastructure that has to be present at every stage of the pipeline without coupling the layers. In this section I describe my design responses for each.

Per-stage cost tracking. Every Gemini call in the pipeline goes through an **AsyncLocal**-backed accumulator declared in **Application/Common/Diagnostics/CostBreakdown.cs**. The accumulator is request-scoped, so parallel scoring tasks within the same incoming HTTP request share a single record while concurrent requests do not interfere with one another. For each stage it tracks the number of calls, the elapsed wall-clock time, the input and output token counts, and the dollar cost computed from the current Gemini 2.5 Flash unit prices. I deliberately hosted the accumulator in the Application layer rather than in Infrastructure: this allows both the orchestrating Application handler and the concrete Infrastructure services to write into the same accumulator without the Application layer reaching downward into Infrastructure, and it preserves the Clean Architecture dependency direction.

Persistent cost ledger. At the end of each request the accumulator is flushed to a PostgreSQL table named **gemini_cost_log**, modelled by the **GeminiCostLogEntry** entity. The accumulator emits one entry per stage that was touched during the request, and each entry is persisted as a row with columns for the user identifier, the request identifier, the request kind (for example **v6_search**, **cv_normalize**, or **worker_backfill**), the stage name (**judge_batched**, **reason_batched**, **skill_expansion**, and so on), the number of calls, the wall-clock duration, the input and output token counts, the dollar cost, and the keyword string when applicable. An administrator can query the table directly through **/api/admin/cost**, which returns the data as comma-separated values grouped by an optional dimension. The numbers themselves are reported in Section 5.

Best-effort writes. A failure in the cost-logging path must never break a user response. The implementation in **Infrastructure/Services/CostLogService.cs** catches every database exception and continues, on the principle that a missing log entry is preferable to a failed search. Structured logging through **ILogger** at every stage means the loss is visible in the application logs and can be diagnosed after the fact.

Health endpoints. The API exposes two health endpoints. The lightweight **/health** endpoint returns a fixed liveness response without touching any dependency; it is consumed by Caddy and by the Docker container's healthcheck on each interval. The heavier **/health/ready** endpoint performs a database round-trip through **IDatabaseHealthService** and is used during deployment to confirm that the new container is ready to serve traffic before old containers are stopped.

Authentication and rate limiting. The API uses stateless JSON Web Tokens for authentication, configured through the **AddJwtBearer** extension in **Program.cs**. Passwords are hashed with the BCrypt.Net-Next library, which exposes the standard work-factor parameter and is regularly audited. Request rate limiting is implemented as ASP.NET Core middleware inside the API project rather than in Caddy — I kept the Caddy layer deliberately narrow (TLS termination, security headers, request forwarding) so the application has a single source of truth for the policy and can apply per-route limits informed by application context.

Startup guards. I added two production-only startup guards that turn configuration mistakes into immediate, loud failures. The first refuses to start the application unless the database connection string carries one of `SslMode=Require`, `SslMode=VerifyCA`, or `SslMode=VerifyFull`. The second refuses to start unless the `Cors:AllowedOrigins` list is non-empty, which prevents a configuration mistake from leaving the application open to cross-origin calls from any origin. Both guards throw at startup and emit a clear log line; a misconfigured deployment cannot drift into a quietly insecure state.

GDPR cascade delete. User-account deletion is implemented as a transactional cascade in `Application/User/Commands/DeleteCurrentUser/DeleteCurrentUserHandler.cs`. The handler reads the CV file key from the user record, runs a transactional cascade through the database (the user's tracker applications, the relevance-explanation rows keyed on the user's current CV version identifier, and the user-profile row itself), and finally issues a best-effort S3 deletion of the CV file. The database is treated as the authoritative artefact for the right-to-be-forgotten obligation; an orphan blob in the S3 bucket is recoverable via the bucket's lifecycle policy and does not violate the deletion semantics.

Soft-determinism on model calls. Every Gemini call carries a short per-call timeout — eight seconds in the inner scoring-loop reason call, fifteen to twenty-five seconds in the dedicated Composite Judge, batched Judge, and batched reason services — and falls back to a deterministic template if the timeout fires or the response fails schema validation. A misbehaving language-model call therefore cannot block the request loop or produce a non-renderable output.

3.9 Chapter summary

In this chapter I turned the requirements from Chapter 2 into a concrete design. The system is a .NET 8 back-end and a React/TypeScript front-end, hosted on AWS behind Caddy, organised as four Clean Architecture projects whose dependency graph the compiler enforces. The scoring pipeline routes each (CV, vacancy) pair through deterministic sub-scores, an anti-flag penalty, a Gemini-based Composite Judge with a skip-band optimisation, asymmetric safety caps, and a batched bilingual reason-generation step. A four-layer cache hierarchy with version-string invalidation keeps the warm-path cost low and removes the need for coordinated cache flushes when the pipeline evolves. The front-end pairs Zustand and TanStack Query with a Tier-based progressive disclosure of evidence chips and a bilingual user experience. Cross-cutting cost tracking, GDPR cascade deletion, and production startup guards are designed once at the Application layer rather than scattered across the code base, which keeps the layer boundaries intact. Chapter 4 walks through how this design is realised in code.

4 | Implementation

In this chapter I walk through how the design from Section 3 was turned into a working, production-deployed system. The first section documents the development methodology and the conventions that guided the code base. The next three sections walk layer by layer through the Domain, Application, and Infrastructure projects. A fifth section describes the front-end implementation. A sixth section covers deployment, operations, and operating costs. A final section presents two engineering stories — challenges I hit that show how the architecture handled change.

4.1 Development methodology

This section describes the development process behind Vakansio: how I organised the work, what conventions guided the code, what quality measures I applied, and which tools I used.

Iterative versioned development. I developed the project as a single author over several months. Rather than fixed-length sprints, I organised the work around explicit versioned releases of the scoring pipeline. Each version corresponded to an observable change in system behaviour: a rebalance of the scoring weights, the introduction of the global-vocabulary path in version 6.3 that replaced the earlier per-entity skill expander, and several contract-drift fixes documented in Section 4.7. The version string is also the cache-invalidation key from Section 3.3, so each iteration is both an engineering increment and a deployable artefact with a defined effect on the persisted caches.

Prototype-first approach. I grew the system outward from a minimal core. The first working version contained only the deterministic seven-axis sub-scorer and a thin API. I added the Composite Judge stage next, then the batched reason generator, then the front-end tier-based evidence disclosure, then the per-stage cost-tracking subsystem. After each step I validated against real CV–vacancy pairs on the deployed instance before adding the next layer. This kept each change small, so I could roll back quickly when something broke.

Coding standards and conventions. The back-end is written in C# on .NET 8 with nullable reference types enabled across every project. The code base follows the default C# naming conventions — **PascalCase** for public members, **camelCase** for local variables and parameters, **IPascalCase** for interfaces. Each CQRS handler is a single class with exactly one public **Handle** method. Clean Architecture project references are enforced at the build level: **Domain/Domain.csproj** declares zero package references, and the Application project references only Domain. On the front-end, TypeScript is built with **noUnusedLocals**, **noUnusedParameters**, and **noFallthroughCasesInSwitch** enabled, so the build fails on dead code and missed switch cases; ESLint enforces the recommended TypeScript and React-Hooks rule sets. Cross-component contracts that would otherwise drift between files are pinned to single-source-of-truth maps such as **verdictMeta.ts** and **evidenceTier.ts**.

Quality assurance approach. Three quality measures run in parallel.

- **Unit tests** cover the pure Domain rules and the sub-score calculators, where the input–output mapping is deterministic; the suite is detailed in Section 5.
- **Author acceptance tests** — for every observable version, I tested against the deployed instance at dsus1dizgh006.cloudfront.net on a fixed set of CVs and search queries before considering the version shipped.
- **Offline evaluation harness** — separate from the production code path, it computes ranking-quality and reason-quality metrics on an LLM-judged gold set; the methodology and results are reported in Section 5.

The continuous cost-tracking subsystem (Section 3.8) complements the test suite by surfacing performance regressions as cost changes in the **gemini_cost_log** table.

Tooling and workflow. JetBrains Rider was my primary IDE for the back-end. I used PostgreSQL locally during development and through Amazon RDS in production. Docker and Docker Compose containerise the back-end for both local runs and the production deployment described in Section 4.6. The deployment is a manual but reproducible script; a CI/CD workflow is the next operational extension and is named in Section 6.

4.2 Domain layer walkthrough

In this section I walk through the Domain project — the innermost layer of the Clean Architecture decomposition introduced in Section 3.3. The project lives under **Domain/** and contains every artefact that describes the business of candidate–vacancy matching without reference to any external technology. **Domain/Domain.csproj** declares zero package references and no project references, so the layer can be reasoned about and unit-tested in isolation from Entity Framework Core, the Gemini SDK, the AWS adapters, and every other infrastructure concern.

Entities. I defined seven entities under **Domain/Entities/**. **JobVacancy** models a vacancy as it has been extracted from a source, together with its normalised analysis as a **jsonb**-stored payload (the L3 cache layer in Section 3.5). **UserProfile** holds the user record, the pointer to the current CV version in object storage, and the cached normalisation of the CV. **ApplicationTracker** models the per-user record of a job the user has decided to act on. **ScoringCacheEntry** is the row backing the L2 scoring cache, keyed by the composite (CV hash, vacancy identifier, scoring version) triple. **SkillVocabularyEntry** backs the global-vocabulary path introduced in version 6.3. **GeminiCostLogEntry** is the per-call row in the cost ledger from Section 3.8. **SavedUrl** records vacancies the user has manually added by URL.

Value objects and enumerations. Two value objects live under **Domain/ValueObjects/**. **RelevanceScore** wraps a floating-point score together with the pipeline stage that produced it; the constructor rejects values outside $[0, 100]$, and **ToPercent()** renders the score for display. **Salary** wraps a minimum amount, a maximum amount, a currency, and a raw-text fallback for cases where the source did not expose a structured value. Five enumerations live under **Domain/Enums/**: **ApplicationStatus** for the tracker workflow, **JobSource** with seven members covering the six aggregators (Djinni, WorkUa, LinkedIn, Jooble, RobotaUa, DOU) and a **Manual** fallback for user-submitted URLs, **ScoringStage** for the per-stage cost annotations, **SeniorityLevel**, and **WorkFormat**.

Pure scoring rules. The folder **Domain/Scoring/** is the substantive core of the Domain layer. It contains the deterministic rules that operate on already-normalised inputs and produce typed outputs. **CvHasher** computes the SHA-256 hash of a canonical projection of the CV-summary JSON — the sorted skill lists, the target roles, the seniority, the English level, and the total experience in months — so a non-substantive edit to the CV does not change the L2 cache key. **GenericGapFilter** drops generic terms (“communication”, “teamwork”) from the missing-skill list before the result is shown to the user. **LanguageGapDetector** recognises the case where the vacancy’s required English level strictly exceeds the candidate’s, using a CEFR ranking from A1 to C2. **RoleFamily** and **RoleFamilyDetector** provide the deterministic family routing from Section 3.4. **SubScores** is a **record** that carries the seven per-axis scores; the current weights live in **ScoringServiceV2.cs:138–147** rather than in the record itself, so a rebalance changes only one file. **ScoringResult** aggregates the headline score, the sub-scores, the verdict, and the evidence chips into the single typed output of the scoring pipeline.

Verdict labels and the bilingual reason path. The file **Domain/Scoring/Verdict.cs** ties together two cross-cutting concerns: the categorical verdict label and the bilingual output. The enumeration carries four members — **Mismatch**, **WeakMatch**, **PartialMatch**, and **StrongMatch**. The companion **VerdictExtensions** class exposes **FromScore(double)**, which maps a composite score to a verdict at the cuts 0.25, 0.50, and 0.75; these are user-interface display cuts and are deliberately distinct from the Composite Judge calibration anchors. The same class exposes **ToEnglishText()**, **ToUkrainianText()**, and **ToShortName()** the first two carry the user-visible bilingual labels (for example, “Strong match” and “Сильна відповідність”), while the third carries the short label that the front-end **verdictMeta.ts** map keys on. Keeping all three vocabularies in the Domain layer means that any change to the verdict-label contract is a single-file edit. Listing 2 shows the core of the file.

```
public enum Verdict { Mismatch, WeakMatch, PartialMatch, StrongMatch }

public static class VerdictExtensions {
    public static Verdict FromScore(double s) => s switch {
        >= 0.75 => Verdict.StrongMatch,
        >= 0.50 => Verdict.PartialMatch,
        >= 0.25 => Verdict.WeakMatch,
        _      => Verdict.Mismatch,
    };

    public static string ToShortName(this Verdict v) => v switch {
        Verdict.StrongMatch => "Strong",
        Verdict.PartialMatch => "Partial",
        Verdict.WeakMatch   => "Weak",
        Verdict.Mismatch    => "Mismatch",
    };
}
```

Listing 2: **Domain/Scoring/Verdict.cs** — the verdict enum and the short-name extension that the front-end keys on.

Interfaces declared in the Domain layer. Two folders carry interface declarations. **Domain/Interfaces/Repositories/** contains seven repository contracts — **IJobVacancyRepository**, **IUserProfileRepository**, **IApplicationRepository**, **ISavedUrlRepository**, **IScoringCacheRepository**, **ISkillVocabularyRepository**, and **IGeminiCostLogRepository**. Repository contracts live in Domain rather than in Application because the aggregate they serve is itself a Domain concept. **Domain/Interfaces/Services/** contains three additional contracts that describe domain operations whose implementation is technology-specific but whose intent is not: **IDeduplicationService**, **IJobSourceService**, and **IRelevancePipeline**. Other contracts that are application-level rather than domain-level — for example **ICvFileStorage**, **ICostLogService**, **IScoringService**, **IBatchedJudgeService**, and **IBatchedReasonService** — live under **Application/Common/Interfaces/** and are covered in Section 4.3.

4.3 Application layer walkthrough

In this section I walk through the Application project — the use-case layer of the Clean Architecture decomposition. **Application/Application.csproj** declares exactly one project reference, to Domain, and five package references: **FluentValidation 12.0.0**, **FluentValidation.DependencyInjectionExtensions 12.0.0**, **MediatR 14.0.0**, **Microsoft.Extensions.Caching.Memory 8.0.1**, and **Microsoft.Extensions.DependencyInjection.Abstractions 10.0.0**. There is no reference to Entity Framework Core, the Gemini SDK, or any AWS SDK — every infrastructure-bound dependency lives behind an interface declared either in Domain or in this layer.

Feature-folder CQRS organisation. The use cases are organised under four feature folders — **Jobs/**, **Tracker/**, **User/**, and **Eval/** — each split into **Commands/** for state-changing operations and **Queries/** for read-only operations. Each use case lives in its own subfolder named after the operation, composed of three files: a ***Command.cs** or ***Query.cs** that declares the request DTO and implements **IRequest<TResponse>** a ***Handler.cs** that implements **IRequestHandler<TRequest, TResponse>** with one **Handle** method; and, where input validation is required, a ***Validator.cs** that derives from **AbstractValidator<TRequest>**. Representative examples include **Jobs/Commands/AddManualJobUrl/**, **Jobs/Queries/GetAggregatedJobsV6/**, **Tracker/Commands/AddToTracker/**, and **User/Commands/DeleteCurrentUser/**. The folder layout makes the CQRS split visible at a glance — a reader can answer “what does the system do?” by scanning the folder names alone.

The orchestrator handler. The largest handler in the project is **Jobs/Queries/GetAggregatedJobsV6/GetAggregatedJobsV6Handler.cs**, at roughly one thousand lines of code. It is the entry point for the search-and-score pipeline whose stages are documented in Section 3.4. Despite its size, the handler contains no direct Gemini call, no Entity Framework Core context, and no scraping logic. Every collaborator — the aggregator service, the scoring service, the Composite Judge, the batched reason generator, the skill-vocabulary service, the cost logger — is injected through its interface. The handler’s job is to compose those collaborators in the right order, manage the per-request cost-tracking scope, and assemble the response DTO; the work of calling out to the Gemini API or the database lives in Infrastructure.

Options pattern for tuning knobs. I bind pipeline tuning knobs through the `IOptions<T>` pattern. The first POCO is `Application/Common/Configuration/ScoringOptions.cs`, which exposes `SyncNormalizeTimeoutSeconds` (default 300). The value is read from the "Scoring" section of `appsettings.json` at startup, so retuning the pipeline does not require a rebuild. The same pattern is used for further tuning knobs under the Application layer.

Common interfaces and DTOs. `Application/Common/Interfaces/` declares the abstractions through which the use cases reach out to the rest of the system: `ICvFileStorage`, `ICostLogService`, `IScoringService`, `ICompositeJudgeService`, `IBatchedJudgeService`, `IBatchedReasonService`, `ISkillVocabularyService`, `ICvDomainRouter`, `IVacancyDomainRouter`, and `ICurrentUserService`. `Application/DTOs/` holds the carrier objects that cross the layer boundary into the API project, including `JobVacancyV6Dto`, `RankedJobDto`, `ApplicationTrackerDto`, and `UserProfileDto`. DTOs are independent of the Domain entities they shadow, so a change to a stored entity does not automatically reshape the public API surface.

Cross-cutting MediatR pipeline behaviour. Input validation runs as a MediatR pipeline behaviour rather than as a call inside each handler. `Application/Common/Behaviors/ValidationBehavior.cs` implements `IPipelineBehavior<TRequest, TResponse>` — on every dispatched request, it collects every registered `IValidator<TRequest>` from the DI container, runs them, and throws a `ValidationException` with the accumulated failures if any rule fails. Only if every validator passes does control reach the actual handler. The wiring lives in `Application/DependencyInjection.cs`, whose `AddApplication` extension method registers all MediatR handlers from the current assembly, registers all `AbstractValidator` subclasses, and registers the pipeline behaviour itself as a transient open-generic service.

Per-stage cost accumulator. I implemented the per-stage cost-tracking subsystem from Section 3.8 in `Application/Common/Diagnostics/CostBreakdown.cs` as a static class backed by an `AsyncLocal<Accumulator?>` field, shown in Listing 3.

```

public static class CostBreakdown {
    private const double InputPricePerMillion = 0.30;
    private const double OutputPricePerMillion = 2.50;

    private static readonly AsyncLocal<Accumulator?> _current = new();

    public static IDisposable BeginScope() {
        _current.Value = new Accumulator();
        return new Scope(_current);
    }

    public static void Track(string stage, long ms, int inT, int outT) {
        _current.Value?.Add(stage, ms, inT, outT,
            (inT * InputPricePerMillion + outT * OutputPricePerMillion) /
1_000_000);
    }

    public static IReadOnlyList<StageStats> GetSnapshot()
        => _current.Value?.Snapshot() ?? Array.Empty<StageStats>();
}

```

Listing 3: **Application/Common/Diagnostics/CostBreakdown.cs** — request-scoped, parallel-safe per-stage cost accumulator. Pricing constants are private to this file, so a Gemini-tier price change is a one-line edit.

The orchestrator handler opens a scope at the start of the request through **CostBreakdown.BeginScope()** each Gemini-calling service in Infrastructure reports its stage statistics through **CostBreakdown.Track(...)** and at the end of the request the handler reads the snapshot through **GetSnapshot()** and persists the rows. **AsyncLocal** storage makes the accumulator both request-scoped (two concurrent HTTP requests do not interfere) and parallel-safe within a single request (the up-to-three parallel reason-generation chunks all write into the same accumulator). I put the accumulator in Application on purpose: both the handler and the Infrastructure services need to read and write it without breaking the Clean Architecture dependency direction.

4.4 Infrastructure layer walkthrough

In this section I walk through the Infrastructure project — the layer that translates the abstractions declared in Domain and Application into concrete calls against the database, the Gemini API, the AWS services, and the seven external job sources. **Infrastructure/Infrastructure.csproj** declares one project reference, to Application, and a focused set of package references: Entity Framework Core 8.0.10 with the Npgsql and Pgvector providers, AWSSDK.S3 4.0.15, BCrypt.Net-Next 4.0.3, HtmlAgility-Pack 1.12.0, System.ServiceModel.Syndication 8.0.0 (RSS), PdfPig 0.1.14 (PDF), and the Microsoft.Extensions.Hosting and Http packages for the background workers and typed HTTP clients.

Persistence with Entity Framework Core. **Infrastructure/Persistence/** holds the database integration. **AppDbContext** is the single **DbContext**, with one **DbSet** per persisted entity. The companion **AppDbContextFactory** provides the design-time factory used by the EF Core CLI during migrations. **Persistence/Configurations/**

contains six **IEntityTypeConfiguration<T>** classes, one per Domain entity that needs explicit column shaping. **Persistence/Migrations/** contains ten sequential migrations from **20260314_InitialCreate** through **20260528_AddSkillExpansionColumns**. **Persistence/Repositories/** contains seven repository implementations, one per Domain repository contract. **Persistence/Entities/RelevanceExplanation.cs** is the only entity declared in Infrastructure rather than Domain; it carries no invariants and exists purely as a denormalised projection of LLM-generated explanations. The cascade-delete behaviour from Section 3.8 is implemented through **OnDelete(DeleteBehavior.Cascade)** annotations in **UserProfileConfiguration** and reinforced by **DeleteCurrentUserHandler** in Application.

Job-source adapters and aggregation. **Infrastructure/JobSources/** contains the seven adapters that pull vacancies from the external sources, split by access pattern. Two are REST API clients (**JoobleApiService**, **RobotaUaApiService**) under **JobSources/Api/**. One is an RSS feed parser (**DouRssFeedService**) under **JobSources/Rss/**. Four are HTML scrapers (**DjinniScraperService**, **LinkedInGuestService**, **WorkUaScraperService**, **ManualUrlScraperService**) under **JobSources/Scraping/**. Every adapter implements the same **IJobSourceService** contract declared in Domain. **JobAggregation/JobAggregationService** orchestrates the request — it resolves every registered **IJobSourceService**, fans the call out in parallel, and catches per-source exceptions so that a single failing source does not break the response (Listing 4).

```
var tasks = _sources.Select(async src => {
    try {
        return await src.SearchAsync(keywords, location, ct);
    } catch (Exception ex) {
        _logger.LogWarning(ex, "Source {Source} failed; continuing",
            src.SourceName);
        return Enumerable.Empty<ScrapedVacancyDto>();
    }
});
var perSource = await Task.WhenAll(tasks);
var all = perSource.SelectMany(x => x).ToList();

_cache.Set($"scrape:{keywords.ToLower()}:{location.ToLower()}", all,
    new MemoryCacheEntryOptions {
        AbsoluteExpirationRelativeToNow = ScrapeCacheTtl, // 30 minutes
        Size = 1,
    });
```

Listing 4: **JobAggregationService** — parallel fan-out with per-source isolation and the L4 scrape cache write (excerpt).

The combined result then passes through **Deduplication/DeduplicationService** and is cached in **IMemoryCache** under the key **scrape:{lowercased keywords}:{lowercased location}**. The thirty-minute TTL is a static readonly field at **JobAggregationService.cs:37**.

Scoring pipeline and normalisation modules. **Infrastructure/RelevancePipeline/V2/Scoring/** implements the pipeline whose stages I designed

in Section 3.4. **ScoringServiceV2** is the implementation of **IScoringService** the weight table at lines 138–147 is the canonical source of the seven sub-score weights. **SubScoreCalculators/** contains the seven per-axis calculators, all stateless and registered as singletons. The deterministic safety layer is **AntiFlagEvaluator** and **ScoringCapService** the latter applies the 0.20 floor, 0.88 ceiling, and the asymmetric caps from Section 3.4. The LLM-bound services are **GeminiCompositeJudgeService**, **GeminiBatchedJudgeService**, and **GeminiBatchedReasonService** **JudgePromptCore** holds the prompt body and the version string that participates in the cache key. The CV-normalisation and vacancy-normalisation folders under **RelevancePipeline/V2/** follow the same five-component shape — keyword router, module resolver, per-domain modules, prompt builder, post-processor. The CV side ships two modules (Tech, Generic); the vacancy side ships six (Tech, HR, Healthcare, Marketing, Sales, Generic). Adding a new domain is a four-step procedure: extend the domain enum, implement the module interface, register the module in **DependencyInjection.cs**, and extend the keyword router.

Adapters, ML API, workers, and the composition root. **Infrastructure/Services/** holds three small adapters:

- **S3CvFileStorage** — AES-256 server-side encryption, pre-signed URLs with five-minute lifetimes, key prefix `users/{userId:N}/cv/{timestamp}-{filename}`. It is registered conditionally on the presence of **S3:CvBucket NoOpCvFileStorage** is the boot fallback for development.
- **CostLogService** — best-effort writes to `gemini_cost_log`. Every database exception is caught and logged.
- **DatabaseHealthService** — the database round-trip used by the `/health/ready` endpoint.

Infrastructure/MlApi/ wraps an optionally deployed Python ML service: **MlApiScoringService** (bi-encoder), **MlApiEmbeddingService** (multilingual-e5-base), **MlApiReasoningService** (Qwen2.5-3B-Instruct), and **MlApiFactualityService** (MiniCheck, used by the reason-quality evaluation in Section 5). **Infrastructure/Workers/** contains four **BackgroundService** implementations: **JobEmbeddingWorker**, **CvSummaryWorker**, **VacancyAnalysisWorker**, and **ReasoningWorker**. All four are flagged and default off in production. **ReasoningWorker** is intentionally not registered at all because the inline batched reason generator superseded its role. The hot path replaces the disabled background normalisation with the synchronous fan-out documented above, bounded by **SyncNormalizeTimeoutSeconds**. The composition root **Infrastructure/DependencyInjection.cs** wires every contract above to its implementation through an explicit registration block — no MEF-style assembly scan — and configures **AddMemoryCache(opts => opts.SizeLimit = 1000)** as the shared in-memory cache backing both the thirty-minute scrape cache and the five-minute response cache from Section 3.5.

4.5 Frontend implementation details

In this section I turn the front-end design from Section 3.6 into a concrete file map and document the implementation choices that the design section did not cover. The front-

end stack is React 19.2.5 with TypeScript 6.0.2 and the Vite 8.0.10 bundler. Routing across the five pages — Login, Register, Profile, Job Feed, and Tracker — is handled by react-router-dom 7.14.2. The production build runs as **tsc -b && vite build**, so a TypeScript compilation error fails the build before Vite ever produces a bundle. Linting runs as **eslint** . under the configuration described in Section 4.1.

Source-tree organisation. **src/components/ui/** holds six design-system primitives, and **src/components/jobs/** holds twelve feature-level components, including the single-source-of-truth maps **verdictMeta.ts** and **evidenceTier.ts** introduced in Section 3.6. **src/pages/** contains one subfolder per route. **src/api/** contains six files: the shared axios client and five per-feature wrappers (**authApi.ts**, **jobsApi.ts**, **profileApi.ts**, **trackerApi.ts**, **userApi.ts**). **src/store/** holds the two Zustand stores. **src/i18n/** holds the internationalisation layer. **src/persistence/** holds the five-file persisted-cache subsystem. The remaining folders are **src/hooks/**, **src/styles/**, **src/types/**, and **src/assets/**.

API client and JWT handling. **src/api/client.ts** declares a single **axios.create()** instance shared by every per-feature wrapper, with request and response interceptors handling JWT injection and the 401 redirect (Listing 5).

```
export const client = axios.create({
  baseURL: import.meta.env.VITE_API_BASE_URL ?? "http://localhost:5180/api",
  timeout: 300_000, // 5 min: cold v6 search can take up to 2 min
});

client.interceptors.request.use(cfg => {
  const token = localStorage.getItem("token");
  if (token) cfg.headers.Authorization = `Bearer ${token}`;
  return cfg;
});

client.interceptors.response.use(r => r, (err) => {
  if (err.response?.status === 401 && location.pathname !== "/login") {
    ["token", "userId", "email"].forEach(k => localStorage.removeItem(k));
    location.replace("/login");
  }
  return Promise.reject(err);
});
```

Listing 5: **src/api/client.ts** — shared axios instance with JWT injection and unauthenticated-redirect handling.

Vite dev-mode proxy. **vite.config.ts** configures **server.proxy** to forward every request under **/api** to **https://api.vakansio.online** with **changeOrigin: true**. The browser only ever issues same-origin requests to **localhost:5173/api/*** Vite forwards them server-side to the production API. Two practical consequences follow. First, the production **Cors:AllowedOrigins** allow-list (Section 3.8) does not need to include the developer's local host, because no cross-origin request ever leaves the browser. Second, the development iteration loop runs against real production data and the real production scoring caches.

State management implementation. The split designed in Section 3.6 maps onto two Zustand stores and the TanStack Query layer. `src/store/authStore.ts` holds authentication state — the JWT, the user identity, and the helpers that read or write the persisted `localStorage` keys. `src/store/jobStore.ts` holds the cross-cutting search state: the current `searchParams` object (`keywords`, `location`, and the `runRelevancePipeline` flag), together with the cached server payload (`jobs`, `totalCount`, `duplicatesRemoved`) and the loading and error flags. Server data also lives in TanStack Query (`@tanstack/react-query 5.100.3`) under its own keyed cache, and the front-end never copies that cache into the Zustand stores. Strictly local state, such as the identifier of the currently open vacancy drawer (`drawerJob`), remains in component-level `useState` in `JobFeedPage.tsx` because no other component reads it.

Persistence subsystem implementation. `src/persistence/` contains five files, one per concern of the design from Section 3.6. `queryPersister.ts` builds an async-storage persister from `@tanstack/query-async-storage-persister` over an IndexedDB store backed by `idb-keyval 6.2.1`. `src/main.tsx` then wraps the application in `PersistQueryClientProvider` and passes the persister together with the cache-buster string (`v6.7.2` at the time of writing). Advancing the buster string causes the persist-client library to drop the persisted blob on the next mount. `migration.ts` performs a one-shot migration from the legacy `localStorage`-backed cache blob to the new IndexedDB key on startup and exposes a `clearAllQueryCache()` helper used during logout. `multiTabSync.ts` opens a `BroadcastChannel` so that a write in one tab invalidates the corresponding query in every other open tab. `storageQuota.ts` exposes three diagnostic helpers — `isStorageAvailable` (IndexedDB-availability probe), `getStorageEstimate` (wrapper over `navigator.storage.estimate()`), and `isQuotaExceededError` (error classifier) — that the rest of the subsystem uses for diagnostics and graceful degradation; it does not gate writes. `cacheEviction.ts` implements the fifteen-megabyte hard cap by dropping the persisted blob in its entirety (`HARD_LIMIT_BYTES = 15 * 1024 * 1024`) once the threshold is crossed, rather than evicting individual entries.

Internationalisation implementation. `src/i18n/` implements the bilingual user experience. `LanguageContext.tsx` provides the React context that propagates the active language down the tree. `useT.ts` exposes a hook that resolves a translation key against the active locale table. `usePlural.ts` handles the plural-form rules for English and Ukrainian, which differ on the cardinality boundaries. `translations.ts` declares the type-safe key catalogue, and the two locale tables (`locales/en.ts` and `locales/uk.ts`) carry the actual strings. No user-visible string is hard-coded in a component file.

4.6 Deployment, operations, and operating costs

In this section I document how the topology designed in Section 3.7 is built, shipped, and run, and what it costs to keep the system in production. The build artefact is a single Docker image produced from the repository root; the production environment is a single Amazon EC2 host that runs two containers through Docker Compose; the deployment is a manual but reproducible procedure.

Container build. `Dockerfile` is a three-stage build that produces a slim, multi-architecture runtime image (`linux/amd64` and `linux/arm64`). The first stage, `sdk-restore`, copies

only the four `.csproj` files into the build context and runs `dotnet restore` the layer cache therefore only invalidates when the package graph changes. The second stage, `sdk-build`, copies the rest of the source and runs `dotnet publish -c Release --no-restore /p:UseAppHost=false` into `/app/publish`. The third stage starts from the official `mcr.microsoft.com/dotnet/aspnet:8.0` image, installs `curl` for the container health probe, creates a non-root `app` user, copies the published output, exposes port 8080, and declares a container-level `HEALTHCHECK` that polls `/health` every thirty seconds.

Compose topology. `docker-compose.production.yml` defines two services on a private bridge network `vacancies-net`. The `caddy` service runs the Caddy 2.8-alpine image with ports 80, 443, and 443 UDP exposed to the host, and two persistent volumes carry the Let's Encrypt account material and configuration. The `api` service runs the `vacancies-api:latest` image with port 8080 exposed only inside the network. A memory cap of seven hundred megabytes protects the t3.micro host (which has roughly nine hundred megabytes of usable memory) from a runaway .NET allocation. The environment variable `BackgroundWorkers__EnableVacancyAnalysis=false` explicitly disables the vacancy-analysis worker; the compose file's inline comment records the rationale that an always-on background worker would burn three to six dollars per day in Gemini calls even with no user traffic. Caddy's `depends_on: api { condition: service_healthy }` ensures the reverse proxy does not start serving traffic before the API container reports healthy.

Caddy reverse proxy. The `Caddyfile` binds the public hostname from `API_HOSTNAME` (the deployed instance uses `api.vakansio.online`) and obtains a Let's Encrypt certificate automatically (Listing 6).

```

{$API_HOSTNAME} {
  header {
    Strict-Transport-Security "max-age=31536000; includeSubDomains"
    X-Content-Type-Options    "nosniff"
    X-Frame-Options           "DENY"
    Referrer-Policy           "strict-origin-when-cross-origin"
    -Server
  }

  handle /health      { reverse_proxy api:8080 }
  handle /health/ready { reverse_proxy api:8080 }

  handle {
    reverse_proxy api:8080 {
      header_up X-Real-IP {remote_host}
      header_up X-Forwarded-For {remote_host}
      header_up X-Forwarded-Proto {scheme}
      header_up X-Forwarded-Host {host}

      transport http {
        read_timeout      600s
        write_timeout      600s
        response_header_timeout 600s
      }
    }
  }

  encode zstd gzip
}

```

Listing 6: **Caddyfile** — TLS termination, hardening headers, and the long timeouts that accommodate cold Gemini calls.

Rate limiting and the CORS policy are intentionally not configured in Caddy; both live in the API project, so the policy has a single source of truth.

Deployment procedure. Deployment is a manual but reproducible script. On the developer workstation I build the image (**docker build -t vacancies-api:<sha> -f Dockerfile .**), save and compress it (**docker save vacancies-api:<sha> | gzip > image.tar.gz**), and copy it to the production host with **scp**. On the host I load the image (**docker load < image.tar.gz**), retag it to **latest**, and roll with **docker compose -f docker-compose.production.yml up -d**. No container registry is involved. The front-end deployment compiles the bundle with **npm run build**, uploads it to the static-hosting bucket with **aws s3 sync dist/ s3://...**, and issues a CloudFront invalidation for the affected paths. The repository has no **.github/workflows/** directory; a CI/CD pipeline is recorded as future work in Section 6.

Health probes and rollout safety. The two-tier health endpoint design from Section 3.8 is realised here. The Docker container's own **HEALTHCHECK** polls **/health** (liveness only, no database touch) with **curl** every thirty seconds, with a five-second timeout, three retries, and a thirty-second start period. Caddy refuses to start until that probe reports healthy through **depends_on: service_healthy**. The deeper **/health/ready** endpoint performs

the database round-trip and is the probe to use when manually confirming that a freshly rolled API container is ready to serve traffic.

Operating-cost analysis. The operating cost decomposes into a fixed infrastructure component and a variable language-model component. At publicly listed June 2026 prices in the **eu-central-1** region, a single t3.micro EC2 instance under on-demand Linux billing costs approximately \$0.0114 per hour (\$8.30 per month). A db.t3.micro RDS instance running PostgreSQL in a single Availability Zone costs approximately \$0.020 per hour (\$14.60 per month), with an additional \$2.30 per month for twenty gigabytes of general-purpose SSD storage. The S3 bucket for CV files draws a per-gigabyte-month charge of about \$0.0245; for the prototype scale, the monthly bill rounds to a few cents. The CloudFront-fronted front-end bucket falls entirely within the always-free tier of one terabyte of transfer and ten million requests per month. SSM Parameter Store charges nothing for Standard-tier parameters at the volumes used by Vakansio. The total fixed monthly infrastructure cost is therefore approximately \$25 to \$28 per month once any Free-Tier allowance is exhausted; while the account remains within the Free-Tier window, the realised cost is near zero.

The variable component is the Gemini 2.5 Flash usage, priced at \$0.30 per million input tokens and \$2.50 per million output tokens at the short-context tier. These two rates appear verbatim as the private constants **InputPricePerMillion** and **OutputPricePerMillion** in **CostBreakdown.cs:20-21** (Listing 3), so the application’s own cost-estimation arithmetic agrees with the published price list. Per-request cost depends on the pipeline path taken (cold versus warm, judge skip-band hit rate, number of pairs that enter the batched reason generator), and is measured continuously through the **gemini_cost_log** table. Per-stage measured numbers and aggregate per-request statistics are reported in Section 5. Three architectural decisions have the most direct effect on the variable cost: the Composite Judge skip-band (avoids 30–40% of judge invocations), the global-vocabulary path (replaces per-entity skill expansion with a single batched call), and the disabled background workers (keep idle Gemini spend at zero).

Operational characteristics. Three properties of the production deployment are worth recording as deliberate choices. The system runs on a single t3.micro host because that fits the cost envelope of a single-author capstone; the migration path to an ECS task definition behind an Application Load Balancer is mechanical from this code base if scale demands it (Section 3.7). The deployment is a manual but reproducible Docker Compose script; a CI/CD workflow is the next obvious operational extension. The four background workers default off in production because their always-on Gemini cost is not justified at current usage; the synchronous fan-out replaces them on the request path, which keeps cost predictable.

4.7 Selected engineering narratives

In this section I report two engineering challenges that I encountered during the implementation of Vakansio and resolved without weakening the layer separation. Each narrative illustrates a different way the architecture absorbed unexpected change.

Frontend-back-end contract drift fix (v6.7.8). The most consequential bug I encountered during the second half of the project was a silent empty-result failure on the front-end search page after a routine back-end serialisation change. The change itself was a single line at **API/Program.cs:66**: the addition of **new JsonStringEnumConverter()** to the JSON serialiser's converter collection, so that the API would emit enum members by name ("**Djinni**", "**StrongMatch**") instead of by their underlying ordinal values. The motivation was Swagger readability and the front-end's preference for typed string unions over ordinal magic numbers.

The change broke the front-end in two related but independent places, both fixed under v6.7.8. The first regression was at **frontend/src/pages/JobFeed/JobFeedPage.tsx:153–157**: the source-filter **<select>** used numeric option values ("**0**", "**1**") and read **Number(sourceFilter)** to compare against the response payload. After the back-end switch the payload carried strings; **Number("Djinni")** returned **NaN** and every **job.source !== sourceFilter** comparison was vacuously truthy. The filter rejected every job, and the user saw an empty list with no error. The fix changed option values to mirror the back-end string names and removed the **Number()** coercion. The second regression involved the verdict-label vocabulary. The Domain enum **Verdict** (Section 4.2) defined four members whose CLR names are **StrongMatch**, **PartialMatch**, **WeakMatch**, and **Mismatch** the string-enum converter therefore emitted these long names on the wire, but **verdictMeta.ts** keyed display metadata on the short forms (**Strong**, **Partial**, **Weak**, **Mismatch**). I resolved this in Domain rather than at the boundary by adding **VerdictExtensions.ToShortName()** at **Verdict.cs:69–75** (Listing 2); the DTO uses **ToShortName()** whenever it serialises a verdict, while the reasoning prompt continues to use **ToString()** because it expects the long names. Two callers, two vocabularies, both routed through a single file. The lesson: contract-changing edits need a cross-stack audit, not a unit test on one side. Single-source-of-truth maps such as **verdictMeta.ts** and **evidenceTier.ts** (Section 3.6) and exhaustive extension methods such as **VerdictExtensions** are what made the second fix a one-file edit rather than a scattered touch-up.

Graceful degradation as an architectural pattern. The third narrative is not a single bug but a recurring pattern. I built the v6 pipeline on the assumption that any single external call can fail or run out of time, and that the appropriate response is rarely to fail the entire user-facing request. The clearest example is the **SkippedNoAnalysis** field on **GetAggregatedJobsV6Result**. A cold v6 search can return up to roughly one hundred and fifty freshly scraped vacancies, each of which needs a Gemini vacancy-normalisation call before scoring can run. The synchronous fan-out is bounded by **ScoringOptions.SyncNormalizeTimeoutSeconds** (currently three hundred seconds). When the budget blows, the remaining un-normalised vacancies are not scored; they are counted into **SkippedNoAnalysis** and returned alongside the partial result set. On a subsequent request from the same user, the still-unanalysed vacancies in the database are picked up and normalised inside the next budget window. The XML documentation on **ScoringOptions** records this explicitly as a “self-healing” property of the pipeline.

The same pattern reappears in three other places. **JobAggregationService** (Listing 4) wraps every individual **IJobSourceService** call in a try-catch so that a single failing

source — most often LinkedIn returning a rate-limit response — does not break the entire aggregated search. Every Gemini call carries a short per-call timeout — eight seconds in the inner scoring-loop reason call, fifteen seconds in the Composite Judge, eighteen seconds in the batched Judge, and twenty-five seconds in the batched reason service — and falls back to a deterministic template if the timeout fires or the response fails schema validation. **CostLogService** catches every database exception on the principle that a missing log entry is preferable to a failed user response. The lesson: at design time, ask not whether each external call will fail, but how the system will behave when it does. Recording the chosen answer in a typed surface such as **SkippedNoAnalysis** makes the degradation observable rather than silent.

4.8 Chapter summary

In this chapter I turned the design from Chapter 3 into a deployed, working system. The back-end is four .NET projects whose dependency graph the compiler enforces, with the Domain layer free of any external package. The Application layer hosts a request-scoped, parallel-safe cost accumulator that both the orchestrating handler and the Infrastructure-bound LLM services write into without breaking the Clean Architecture dependency direction. The Infrastructure layer wires seven job-source adapters, the four-stage Gemini scoring pipeline, S3 and SSM Parameter Store adapters, and four flag-gated background workers (all default-off in production). The front-end pairs Zustand for cross-cutting UI state with TanStack Query for server data, persists the query cache to IndexedDB with a fifteen-megabyte hard cap and a cache-buster version, and renders verdicts and explanations bilingually through a typed translation layer. The deployment is a manual but reproducible Docker Compose roll-out behind Caddy on a single EC2 t3.micro host, with a fixed monthly infrastructure cost of approximately \$25–\$28 once the Free-Tier window closes. Two engineering narratives — a v6.7.8 contract drift fix and a graceful-degradation pattern — illustrate how the architecture absorbed unexpected change without scattering the response across the layers. Chapter 5 reports the measurements that close the loop on these im

5 | Validation

In this chapter I document how I evaluated the Vakansio pipeline and what the evaluation found. The chapter has two equally important aims. The first is to describe a solid evaluation approach — split into independent layers, each measured with the right technique for its data type — that another engineer could rerun and extend. The second is to report the findings honestly. The primary result on ranking quality is inconclusive at the conventional significance level on the gold set available within the project’s scope. The decomposition follows the Holistic Evaluation of Language Models framework [6] and uses an Anthropic Claude judge where no reference answer exists to compare against [7].

5.1 Evaluation philosophy and methodology pivot

In this section I explain how my evaluation strategy reached its current shape. My first design used one end-to-end metric — score MAE — as the main evidence for the claim that the Composite Judge improves match quality over a linear weighted sum of the sub-scores. Two independent audits found three structural problems with that metric, so I revised the design before collecting the final results. The revised design keeps the score-error metric as a calibration diagnostic and uses top-k ranking quality against independent-vendor LLM ratings as the new primary endpoint.

Why I demoted the original score-error metric. The score-error harness compares the pipeline’s output to a deterministic ideal computed from gold inputs. The audits found three problems with this comparison:

- **Cap asymmetry.** The pipeline applies safety caps and an anti-flag penalty (Section 3.4); the ideal does not. Any pipeline that ships caps is mechanically penalised on this metric.
- **Selection circularity.** The gold pair set was split on three signals — title overlap, seniority match, and skill overlap — that are the very inputs the Linear pipeline uses. The test set is engineered to favour the Linear baseline.
- **Composite Judge over-penalisation.** The Judge prompt has rule-precedence clauses that allow it to override its bounded-adjustment band on under-qualification, language gap, or family mismatch, and the post-judge clamp does not enforce the bound. This is a real calibration issue, but it is more useful as a per-axis diagnostic than as the headline result.

The revised primary endpoint. I adopted top-ten NDCG against independent-vendor relevance ratings as the new primary evidence. The rater is from Anthropic’s Claude family; the system under evaluation runs on Google’s Gemini. Using a different vendor avoids the circular reasoning the original harness had. A full multi-vendor ensemble — adding judges from a second non-Anthropic family — would strengthen the protocol further and is recorded as future work in Section 5.7. My protocol follows Zheng et al. [7]: anchor-only ordinal ratings, randomised candidate order, mandatory per-rating rationale, and stratified batches across verdict buckets. The same protocol underpins the rubric-based evaluation of the bilingual reason text in Section 5.5.

What this chapter reports. I organise the remainder of the chapter around the five evaluation layers that emerged from this design. Section 5.2 describes the layered decomposition. Section 5.3 summarises the two structured-output normalisation layers that I froze before the methodology pivot. Section 5.4 reports the score-error metric as a calibration diagnostic, including the per-axis bias decomposition. Section 5.5 reports the reason-text-quality layer, including the complete prompt iteration cycle the eval pipeline guided through four versions. Section 5.6 reports the primary endpoint on top-ten ranking quality, including the per-curriculum-vitae variance behind the headline confidence interval. Section 5.7 names the limitations of the evaluation and the specific extensions that would resolve each. A worked example tracing a single (curriculum vitae, vacancy) pair through every layer is in Section A.

5.2 Layered evaluation architecture

The pipeline produces four things per (CV, vacancy) pair: a structured CV summary, a structured vacancy analysis, a numeric score, and a bilingual reason text. My main choice: evaluate each one with a method that fits its data type, instead of squashing everything into one end-to-end number. A single metric blurs failures across stages and makes fixes harder. My decomposition follows the Holistic Evaluation of Language Models framework [6]. The five layers I use are the road map for the rest of the chapter:

- **Layer 1 — CV normalisation.** Structured JSON, graded against my own reference normalisations. Headline: per-field F1 = 0.896 on 25 CVs. Frozen after nine prompt versions.
- **Layer 2 — Vacancy normalisation.** Structured JSON, graded against my own reference normalisations. Headline: per-field overall = 0.810 on 355 of 357 vacancies. Frozen after twelve prompt versions.
- **Layer 3 — Score.** Numeric scalar in $[0, 1]$ compared to a computed ideal from gold inputs. Headline: Linear MAE = 0.101, Composite MAE = 0.187 on 392 pairs. Kept as a calibration diagnostic (see Section 5.4).
- **Layer 4 — Reason text.** Bilingual natural language, judged by Claude Opus 4.7 on a six-dimension rubric. Headline: overall rubric mean = 8.35 / 10 on 391 pairs. Prompt v6 wins +0.30 over the v4 baseline.
- **Layer 5 — Ranking.** Top-ten ordered list, judged by Claude Sonnet 4.6 on a single match-quality score. Headline: Δ NDCG@10 = -0.047 , 95% CI $[-0.168, +0.065]$ on 14 CVs. Inconclusive at $\alpha = 0.05$.

Each layer owns its own failures. A misclassified vacancy is a Layer 2 problem regardless of the score. A weak reason on a correctly scored pair is a Layer 4 problem. A stable score but a worse top-ten ordering is a Layer 5 problem. Each layer also uses the method the literature treats as standard for that data type: hand-annotated references with field-level scoring for the two structured layers, a computed ideal for the numeric layer, LLM-as-Judge on a rubric for the natural-language layer [7], and ordering-quality metrics (NDCG, Spearman, Kendall) against per-pair relevance ratings for the ranking layer [5]. The same separation Clean Architecture imposes on the production code (Section 3.3) organises the evaluation: the Domain types the scoring service consumes are exactly the inputs my eval tool feeds to each layer. The eval tool, a separate .NET project under **EvalTool**/, runs

offline against snapshots of the gold data and shares the production interfaces, so it tests the same code paths that ship to users.

5.3 Layers 1 and 2 — frozen normalisation baselines

In this section I report the two structured-output normalisation layers, both of which I closed before the methodology pivot in Section 5.1 and against whose frozen state the later layers compose. Both layers follow the same shape: a gold set of author-curated reference normalisations, a battery of per-field graders applied to the pipeline output, and an append-only history journal that records every iteration of the prompt or post-processor.

Layer 1 — curriculum-vitae normalisation. The gold set comprises twenty-five curricula vitae, which I normalised into the same JSON shape that the production pipeline persists as `UserProfile.CvSummary`. Nineteen field-level scored metrics are computed by the graders declared in `EvalTool/Grading/EvaluationEngine.cs`: F1 for skill list comparisons (`technical_skills`, `domain_skills`, `unverified_skills`), Jaccard overlap for target-role lists, exact-match for categorical fields (seniority, work format), and an ordinal-distance grader on the Common European Framework of Reference ladder (A1–C2) for English level. The headline number is the macro-averaged score across these metrics. The current production CV-normalisation service reaches an overall score of 0.896 on this gold set after nine prompt versions; I preserve the iteration history in `eval_history/runs.csv`. I consider the layer frozen because subsequent runs change the headline number by less than the per-iteration noise observed across the last three iterations.

Layer 2 — vacancy normalisation. The gold set comprises three hundred and fifty-seven vacancies I sampled across the seven source adapters and stratified to cover four heuristic strata (Strong, Partial, Weak, Mismatch) that align with the Composite Judge verdict bucket names. I normalised each vacancy into the same JSON shape that the production pipeline persists as `JobVacancy.VacancyAnalysisJson`, and seventeen field-level scored metrics are computed by the graders declared in `EvalTool/Grading/VacancyEvaluationEngine.cs`, adapted to the vacancy schema (must-have skill lists, nice-to-have skill lists, domain, seniority, work format, language requirement). The current production vacancy-normalisation service reaches an overall field-weighted score of 0.810 across three hundred and fifty-five of the three hundred and fifty-seven gold vacancies (two were excluded by the loader because of malformed source records) after twelve prompt versions; the iteration history is preserved in `eval_history/vacancy/runs.csv`. I consider the layer frozen on the same noise-threshold criterion as Layer 1.

What the frozen baselines fix, and what they do not measure. Because Layers 1 and 2 are frozen, all the later layers run against a known, reproducible upstream. I report Layer 3 and Layer 5 results both against the production normalisation services and (in parallel) against the gold normalisations directly, which gives an upper bound on what the pipeline could reach with perfect upstream. Layer 4 is reported only against the production path because reason text is generated after scoring. Two caveats. First, per-field F1 measures how well the pipeline matches my own gold annotation — not how well the annotation matches reality. Both gold sets were curated by a single annotator (me), and I did not collect an inter-annotator coefficient. Second, the fields with lowest F1 — the skill lists,

between roughly 0.47 and 0.62 — are the same fields the scoring formula leans on hardest. I come back to both points in Section 5.7.

5.4 Layer 3 — score as a calibration diagnostic

In this section I report the score-error layer and explain why I keep it as a calibration diagnostic rather than as the primary evidence for the thesis claim. The layer measures how closely the pipeline’s end-to-end composite score tracks a deterministic ideal computed on gold inputs. The harness was my original primary endpoint; the methodology pivot in Section 5.1 moved it to a diagnostic role after two independent audits found three structural issues that make the metric a poor fit for the question the thesis actually asks.

The harness. For every (curriculum vitae, vacancy) pair in a fixed three-hundred-and-ninety-two-pair test set, the harness loads the gold CV summary and the gold vacancy analysis, runs the seven sub-score calculators on those gold inputs, and clamps the weighted sum to $[0, 1]$. The resulting **ideal_score** is the raw weighted sum with no anti-flag penalty, no missing-must-have soft penalty, no language-gap cap, and no family-mismatch cap. The harness then reads the pipeline’s end-to-end output for the same pair and reports the absolute difference. The harness is implemented in **EvalTool/Grading/ScoringEvaluationEngine.cs** and is run through the orchestrator entry point **ScoringEvalOrchestrator.cs**.

First-look numbers. On the 392-pair test set, the Linear pipeline (weighted sum of the seven sub-scores, no Composite Judge) reaches 0.899 overall — MAE 0.101, verdict-bucket match 67.3 per cent. The Composite pipeline (Judge invoked on every pair) reaches 0.813 — MAE 0.187, verdict-bucket match 31.9 per cent. The naive reading says the Composite Judge is worse than the Linear baseline. The two audits show why that reading is misleading.

Three audit findings (in summary). I summarise the three structural issues here; the full audit decomposition is in **Capstone Project/EVAL_VALIDITY_APPENDIX.md**.

- **Cap asymmetry.** The pipeline applies the safety caps and anti-flag penalty from Section 3.4; the ideal does not. A per-pair decomposition reveals that approximately fourteen per cent of pairs carry an active anti-flag and drift by an average of -0.379 against the ideal, while the remaining eighty-six per cent without an anti-flag drift by an average of -0.056 . Together these account for almost all of the Linear pipeline’s headline error: $0.14 \times 0.379 + 0.86 \times 0.056 = 0.101$, matching the Linear MAE exactly. So the metric measures the caps, not pipeline quality.
- **Selection circularity.** I stratified the 392-pair test set on title overlap, seniority match, and skill overlap — the same primitives the Linear pipeline’s **RoleIntentMatchCalculator** and **SkillMatchCalculator** consume. The Linear pipeline is graded on its home turf. The audit estimates this asymmetry inflates the Linear advantage by 0.02–0.04 MAE points (40–50 per cent of the visible gap).
- **Composite Judge over-penalisation.** The Judge prompt’s rule zero requires output within ± 0.05 of the input linear score, but rules three (under-qualification), four (language cap), and five (different-family cap) explicitly override rule zero when triggered, and the post-Judge clamp in **GeminiCompositeJudgeService.cs** enforces

$[0, 1]$ but not the ± 0.05 band. The five largest Linear-to-Composite divergences are $-0.39, -0.38, -0.26, -0.23, -0.21$, all on pairs where one of the three override rules fired. The fix is straightforward: enforce a hard $|s_{\text{judge}} - s_{\text{linear}}| \leq 0.10$ clamp at the boundary.

What remains after the artefacts are subtracted. Of the visible 0.086 gap between the Linear and Composite headlines, the audits attribute roughly 0.04 to eval-engine artefacts (cap asymmetry plus selection circularity) and roughly 0.05 to the Composite over-penalisation. Subtract both and the Composite Judge would still underperform on the score-tracking metric, but the gap would be about half what the headline suggests. I confirm this empirically in Section 5.6: running the saved Composite scores through a post-hoc ± 0.10 clamp collapses the largest negative outlier from about -0.54 to essentially zero.

Why this metric answers the wrong question. The deeper point: score-tracking is the wrong primary endpoint. The thesis claim is about ranking quality — whether the Composite top-ten is better-ordered against human-judged relevance than the Linear top-ten. A pipeline can have a higher MAE against the linear ideal while still ordering its top ten better. I replaced the primary endpoint; the new endpoint is in Section 5.6. The score-tracking harness stays because the per-sub-score bias breakdown it produces — for example, $+0.028$ on **DomainAlignment** and $+0.045$ on **RoleIntentMatch** — is useful when I rebalance a sub-score calculator.

5.5 Layer 4 — reason-text quality and the prompt iteration cycle

In this section I report the reason-text-quality layer, which evaluates the bilingual English and Ukrainian explanations the pipeline generates for the top-thirty candidate vacancies in every result. The data type here is short natural-language text, so the field-grader method used in Layers 1 and 2 does not work. My methodology is large-language-model judging on an independent Anthropic Claude family on a structured rubric, following the protocol of Zheng et al. [7] and the domain-specific rubric design of Hu et al. for hiring explanations [8].

The judge. My judge model is Claude Opus 4.7 from Anthropic. The choice of an Anthropic judge avoids the within-system bias that a same-family judge would carry: the model whose outputs are being rated is Google’s Gemini 2.5 Flash, so the judge and the system under test come from different vendor families. Opus is the strongest model in the Anthropic family at the time of the evaluation; Zheng et al. recommend using the strongest tier on the judge side in an asymmetric setup. Using a single judge family is a deliberate scope choice, not the final word on method: I record a full multi-vendor ensemble as the principal limitation in Section 5.7.

The rubric. Each (candidate, vacancy) reason text is rated on six dimensions. Four are ordinal on the six-point anchor set $\{0, 2, 4, 6, 8, 10\}$: **factual_accuracy** (do the strengths, gaps, and context claims correspond to the actual CV and vacancy content?), **completeness** (are the top factual signals surfaced?), **specificity** (is the phrasing pair-unique rather than generic boilerplate?), and **relevance_for_user** (is the explanation actionable for an apply-or-skip decision?). Two are binary: **verdict_match** (does the ver-

dict word agree with the score bucket?) and **uk_term_preservation** (are technical brand-and-tool tokens preserved in Latin script in the Ukrainian half?). I record the rubric, the anchor descriptions, and worked examples in **docs/research/reason_eval_rubric.md**.

Bias mitigation in the judging protocol. I apply five mitigations per Zheng et al. I anonymise candidate identifiers at random (candidate A versus candidate B, with a random mapping back to the actual prompt version being tested). I instruct the judge not to compare candidates against one another within a pair, only against the rubric. The ordinal dimensions use anchor-only scoring with no intermediate values. Pairs are organised into batches of twenty-eight stratified proportionally across the four verdict buckets. The judge emits a per-rating rationale alongside each numeric score, for human inspection and downstream auditability.

Does the judge agree with itself? — pilot check. Before running the main ratings, I called the Claude Opus 4.7 judge twice on the same ten pilot pairs, with randomised order and non-zero temperature, and computed Krippendorff’s α across the rubric. This number says how often the judge agrees with itself across two runs — it is **not** a check that two different vendors agree with each other. My threshold was $\alpha \geq 0.6$ for ordinal dimensions and $\alpha \geq 0.7$ for binary ones, slightly below the standard Krippendorff (2004) threshold of 0.667. I used the lower bar because this check is meant to catch a wildly unstable judge, not to formally prove rater reliability. Mean ordinal α on the pilot was 0.695, with binary raw agreement between 93 and 97 per cent. The lowest per-dimension α — **factual_accuracy** at $\alpha = 0.603$ — was 0.003 below the threshold; I report it openly. A real cross-vendor inter-rater check remains the main limitation (Section 5.7).

Author spot-check of the gold ratings. I did not treat the judge ratings as a black box. I picked a stratified sample of pairs across the four verdict buckets and read each one end-to-end — the CV, the vacancy, the judge’s score on every dimension, and the rationale. The point was a sanity check, not a re-annotation. The judge’s ratings still served as the gold values in the statistical inference. On the spot-checked pairs I saw no obvious bias pattern, but I want to be clear: I am the author of the project under evaluation, so I am not a disinterested rater, and I report the spot-check for transparency, not as an independent gold standard.

Statistical inference protocol. For each prompt-version comparison, my headline number is the paired difference of the per-pair overall mean across all six dimensions. Confidence intervals come from paired bootstrap with ten thousand resamples. Per-dimension deltas are reported separately as exploratory signals.

The prompt iteration cycle. I demonstrate a complete detect-fix-validate-iterate loop through four prompt versions, each developed in response to a regression that the previous iteration surfaced. Table 4 summarises the comparison endpoints.

Comparison	Δ overall	95% CI	Significant $\alpha = 0.05$	at	Headline signal
v4 vs v2	+0.55	[+0.42, +0.68]	Yes		Context enrichment works, factual regression detected
v5 vs v4	-0.03	[-0.14, +0.08]	No		Targeted fix introduces a recall trade-off
v6 vs v4	+0.30	[+0.19, +0.40]	Yes		Templates resolve trade-off across all four ordinal dimensions

Table 4: Paired-bootstrap overall reason-quality deltas across the three head-to-head studies. Each study uses approximately 391 pairs, anonymised, with the protocol described in this section.

The per-version story in one sentence each: **v4** added context-lead phrasing and a **ReasonContext** record from **ScoringServiceV2.BuildReasonContext()** with a **ReasonValidator** safety net, gaining +0.55 overall over v2 but regressing **factual_accuracy** by -0.43 when the language model inferred labels from its own judgement; **v5** rewrote the context-lead rule to forbid inference and require strict **ReasonContext** referencing, recovering **factual_accuracy** by +0.71 but regressing **completeness** by -0.62 and **relevance_for_user** by -0.37 as the model became too cautious; **v6** replaced free-form context generation with four constrained templates (Role-Family Mismatch, Cross-Domain, Overqualification, Underqualification) that interpolate literal **ReasonContext** values into fixed sentence shapes, producing significant gains on all four ordinal dimensions (**factual_accuracy** +0.50, **completeness** +0.23, **specificity** +0.77, **relevance_for_user** +0.31, overall +0.30) and is the production version. The iteration cycle itself is a contribution: each detection was a confidence-interval calculation, not a guess, and each fix was a single component change with the rest of the pipeline held still.

5.6 Layer 5 — top-ten ranking quality (the primary endpoint)

In this section I report the primary endpoint of the thesis: top-ten Normalised Discounted Cumulative Gain (NDCG@10) [5] of the Composite pipeline against the Linear baseline, measured on a per-(CV, candidate-set) basis with paired-bootstrap confidence intervals. The endpoint is what the original thesis claim is actually about — whether the Composite pipeline produces a better-ordered top-ten list than the Linear pipeline — and I adopted it after the methodology pivot in Section 5.1.

The judge. For this layer my judge is Claude Sonnet 4.6 from Anthropic, distinct from the Claude Opus 4.7 judge in Section 5.5. The choice of the more cost-effective tier reflects the task: each per-pair rating is a single ordinal value on the anchor set $\{0, 2, 4, 6, 8, 10\}$

rather than the six-dimension rubric Layer 4 requires, and I need the ratings at scale (391 pairs across 14 CVs). The cross-vendor configuration is preserved — an Anthropic judge rates the output of the Gemini-backed Vakansio pipeline — and the same anchor-only ordinal protocol with mandatory per-rating rationale and randomised candidate order is applied. The single-vendor judge limitation is named explicitly in Section 5.7.

The rated quantity. The judge rates each (CV, vacancy) pair on a single dimension named `match_quality`. Anchor descriptions span the range from “no signal of fit between the candidate and the vacancy” at zero to “the candidate is an exceptional match for the role as described” at ten. Pairs are presented to the judge with the full CV summary and vacancy analysis but without either pipeline’s score; the rating is therefore a relevance signal independent of either system under test.

Author spot-check of the match-quality ratings. I subjected the match-quality gold set to the same plausibility audit described in Section 5.5 — a stratified sample across verdict buckets, read end-to-end against the supporting evidence — and recorded no rater-flagged bias pattern. The same absence-of-evidence limitation and the same single-rater interest in the outcome apply.

The endpoint and the test. For each of the 14 CVs, I independently rank the candidate vacancy set (approximately 28 vacancies per CV, stratified across verdict buckets) by the Composite pipeline and by the Linear pipeline, and score the top ten of each against the judge’s match-quality ratings using the standard NDCG@10 definition with $\log_2$ discounting. The per-CV primary statistic is the difference between the Composite and Linear NDCG@10. The aggregate statistic is the mean of the per-CV differences, with a 95 per cent paired-bootstrap confidence interval over the 14 CVs (10,000 resamples).

The headline result. The mean of the per-CV differences is $\Delta \text{NDCG@10} = -0.047$, with a 95 per cent paired-bootstrap confidence interval of $[-0.168, +0.065]$. The interval crosses zero. Cohen’s d on the per-CV differences is -0.199 — a small directional effect favouring the Linear pipeline. The per-CV outcome is five Composite wins, nine Linear wins, no ties. My honest reading: the thesis claim that the Composite Judge improves ranking quality is neither supported nor refuted on this gold set. The data warrant “directionally non-significant” — not “wins”, not “loses”.

Both pipelines rank well on the full ordering. When I look at the full ordering (not just the top ten), both pipelines correlate strongly with the judge’s relevance ratings. Per-CV Spearman correlation is $\rho = +0.690$ for Composite and $\rho = +0.691$ for Linear; Kendall’s τ is $+0.481$ and $+0.503$. Both are far above what a random ranker would produce. The underlying scoring methodology works regardless of whether the Composite Judge stage is enabled, and the two pipelines are statistically indistinguishable on the full ordering too.

The headline hides a big spread between CVs. The Composite pipeline wins on some CVs and loses on others. It wins where the Judge’s qualitative call changes the ordering for the better — a senior QA automation CV ($\Delta = +0.331$), a frontend-engineer CV with React vs. Angular candidates ($\Delta = +0.158$), and my own career-switching product-manager CV ($\Delta = +0.199$). It loses where the deterministic skill match is already strong and the Judge adds noise — a broad full-stack CV where the Judge over-penalises depth

specialisation ($\Delta = -0.643$), an engineering-manager CV ($\Delta = -0.263$), and a senior PM CV ($\Delta = -0.172$). The single broad-full-stack CV alone drives most of the negative aggregate. Remove it and the mean delta drops to about -0.001 — essentially a tie. I keep the outlier in the headline because cutting inconvenient data is itself risky, but I flag it openly here so the reader can judge it.

Sensitivity check under the proposed calibration clamp. The fix from Section 5.4 is a hard ± 0.10 clamp at the boundary of `GeminiCompositeJudgeService`. The clamp is a simple post-processing filter on the Judge’s output. The Composite and Linear scores are already saved per pair, so I apply the clamp arithmetically to those saved scores. There is no need to re-run the Gemini Judge calls: the clamp does not change what the model decides, only the value the boundary returns. I apply it per pair as $s'_{\text{composite}} = \max(s_{\text{linear}} - 0.10, \min(s_{\text{linear}} + 0.10, s_{\text{composite}}))$ and recompute NDCG@10 against the unchanged Sonnet ratings. The sensitivity analysis reproduces the published $\Delta = -0.047$ within rounding as $\Delta = -0.040$. Under the clamp the mean moves to $\Delta = +0.018$, with a 95 per cent CI of $[-0.054, +0.091]$ that still crosses zero. The sign flips from negative to positive, but the verdict — “directionally non-significant” — does not change. The largest outlier, the broad-full-stack CV `17_realformat_swe_fullstack`, drops from about -0.54 to ≈ 0 under the clamp, confirming the audit’s attribution to the rule-precedence over-penalisation. The per-CV outcome shifts from 5–9 to 7–7. I record deploying the fix as future work in Section 6 because production deployment needs a fresh pipeline run; the point reported here is that the methodology’s verdict holds whether the fix ships or not.

What this layer does and does not claim. The layer does not claim that the Composite Judge improves ranking quality. It does not claim the opposite either. The interval crosses zero. What the layer does claim is that the methodology is sound: independent-vendor LLM judging on an anchor-only relevance scale, NDCG@10 with paired-bootstrap intervals, per-CV variance reported openly, and the largest negative case attributed to a fixable calibration defect. The current answer is honestly inconclusive on this gold set. The conditions under which the answer would change are in Section 5.7. A full end-to-end worked example tracing a single (CV, vacancy) pair through all five layers is in Section A.

5.7 Threats to validity

In this section I name the limitations of the evaluation and the specific extensions that would resolve each of them, in order from the most consequential to the least.

CV sample size. The primary endpoint is computed over 14 CVs. The interval $[-0.168, +0.065]$ is wide enough that “directionally non-significant” — neither a clean win nor a clean loss — is the strongest thing the data support. A bigger gold set would either tighten the interval to exclude zero on one side or confirm the per-CV spread. I do not make per-family claims (Engineering, Data, Product, Design) at the inferential level, because no family has the 30+ CVs that a paired test would need; per-family numbers are descriptive only.

Outlier influence on the headline delta. The single broad-full-stack CV (`17_realformat_swe_fullstack`) contributes -0.643 and drives most of the negative aggregate. Remove it and the mean drops to about -0.001 . I keep the outlier in the

headline because removing inconvenient data is itself a hazard, but I flag it openly here so the reader can decide how to weigh it.

Composite Judge calibration defect. The over-penalisation pattern documented in Section 5.4 is the largest identifiable mechanism behind the observed negative cases on Layer 5. A hard $|s_{\text{judge}} - s_{\text{linear}}| \leq 0.10$ clamp at the boundary of **GeminiCompositeJudgeService** would isolate the Composite Judge’s qualitative contribution from this artefact. The fix is documented and the post-hoc sensitivity check in Section 5.6 quantifies its effect: applying the clamp arithmetically moves the mean delta from $\Delta = -0.040$ to $\Delta = +0.018$ and collapses the largest negative outlier from approximately -0.54 to essentially zero. Both pre- and post-clamp confidence intervals still cross zero, so the methodology’s verdict of inconclusive at $\alpha = 0.05$ is robust to whether the fix is deployed. Production deployment is recorded as future work in Section 6.

Single-vendor judge family. All judge ratings on Layers 4 and 5 originate within the Anthropic Claude family — Claude Opus 4.7 on Layer 4 and Claude Sonnet 4.6 on Layer 5. The intra-vendor configuration was a deliberate choice: it allowed a single rating pipeline and prompt format to be reused across the two layers, and the two model tiers match each layer’s task. The inter-vendor extension — adding judges from at least one other vendor family, for example an OpenAI GPT model and a Mistral model — would strengthen the external validity of the Zheng et al. protocol and would let me measure inter-judge agreement directly. A Gemini Pro judge is not a suitable addition because Gemini Pro and the production Gemini 2.5 Flash share the same vendor family. I record the extension as future work in Section 6.

Single-annotator gold sets in Layers 1, 2, and the spot-checks. I curated the CV and vacancy gold normalisations as a single annotator. My author spot-checks of the Layer 4 and Layer 5 ratings are plausibility audits under a non-blind protocol rather than independent annotations, and I am not a disinterested rater. Inter-annotator agreement coefficients on a subset of the gold normalisations and on a subset of the match-quality ratings — by an annotator with no interest in the outcome — would be the appropriate extension; both are recorded as future work in Section 6.

Judge-in-the-loop prompt iteration. I used the Claude Opus 4.7 judge both while developing the Layer 4 prompts and for the final v6-versus-v4 comparison. The risk: v6 might just be better at pleasing one Anthropic checkpoint, not better at helping a human user. My defence is that the v6 prompt’s gains come from hard structural changes — literal field interpolation, fixed sentence shapes, priority ordering — rather than from prose style. The per-dimension wins back this up: factual accuracy improved because the prompt forbids inference, specificity improved because of explicit **vacancy_role_short** interpolation. A human comparison, or one judged by a different vendor, would confirm this and is recorded as future work.

Surface-form sub-score calculators and production A/B. The deterministic sub-score calculators use canonical-alias intersection and Jaccard overlap rather than embedding-based semantic similarity. A vacancy requiring Kubernetes and a CV listing Docker, Helm, and OpenShift score lower on the skill axis than the qualitative match would suggest. The Composite Judge is the design response — the model spots the equivalence — and what the

Linear baseline misses is limited by surface-form matching. Separately, the v6 prompt is deployed to production; an A/B comparison against v4 under real-user click-through rates is recorded as the natural next step. The reason-quality evaluation is a strong predictor of user satisfaction in the cross-vendor judging literature, though it does not measure it directly.

What would change the answer. Three changes, in combination, would be sufficient to resolve the central thesis claim definitively. First, extending the CV gold set to at least thirty per role family would tighten the Layer 5 confidence interval enough to support per-family inference. Second, the Composite Judge calibration fix would remove the largest identifiable confound in the headline. Third, a multi-vendor judge ensemble would establish that the result is not an artefact of the single judge family chosen for the current evaluation. None of the three is conceptually difficult; all three are recorded as future work in Section 6.

5.8 Chapter summary

In this chapter I described the layered evaluation methodology — the result of a mid-thesis pivot away from a structurally biased score-error endpoint — and the empirical findings it produced. Layers 1 and 2 are frozen at per-field scores of 0.896 (25 CVs) and 0.810 (355 of 357 vacancies). Layer 3 stays as a calibration diagnostic; the visible Linear-vs-Composite gap of 0.086 is explained by cap asymmetry, selection circularity, and a documented Composite Judge over-penalisation that has a one-line fix. Layer 4 shows a significant improvement of the v6 prompt over the v4 baseline on all four ordinal rubric dimensions (overall $\Delta = +0.30$, 95 per cent CI $[+0.19, +0.40]$, $N \approx 391$ pairs), judged by Claude Opus 4.7. Layer 5 — the primary endpoint — is inconclusive on 14 CVs (Δ NDCG@10 = -0.047 , CI $[-0.168, +0.065]$, judge Claude Sonnet 4.6); a post-hoc clamp shifts the headline to $+0.018$ without changing the verdict. The methodology stands regardless of the empirical verdict. The conditions under which the answer would change are stated openly. Chapter 6 revisits the four objectives from the introduction and outlines the work that would tighten the answer.

6 | Conclusion

6.1 Project summary

In this thesis I presented Vakansio, a deployed personalised job-matching service for the Ukrainian IT market, together with the evaluation methodology and empirical results behind it. The system pulls vacancies from seven sources, turns them and the candidate's CV into structured data, and scores every (CV, vacancy) pair. The score comes from a deterministic seven-axis sub-score calculator, refined by a Composite LLM Judge, and the result is presented with bilingual English and Ukrainian explanations. The deployed instance runs on a single Amazon EC2 t3.micro host behind Caddy, with managed PostgreSQL, S3 object storage, and CloudFront delivery. It is reachable at dsus1dizgh006.cloudfront.net. The implementation follows the Clean Architecture decomposition of Robert C. Martin [2] with the SOLID principles mapped to concrete code artefacts.

My evaluation breaks the pipeline into five layers — CV normalisation, vacancy normalisation, score, reason text, and ranking — with a method matched to each layer's data type. The decomposition follows the multi-axis principle of the Holistic Evaluation of Language Models framework [6]. The two normalisation layers are frozen at per-field scores of 0.896 (25 CVs) and 0.810 (355 of 357 vacancies). The score layer is reported as a calibration diagnostic after two audits identified structural artefacts in its harness. The reason-text layer reports a significant improvement of the v6 prompt over the v4 baseline on all four ordinal rubric dimensions (overall $\Delta = +0.30$, 95 per cent CI $[+0.19, +0.40]$, $N \approx 391$ pairs), judged by Claude Opus 4.7. The ranking layer is the primary endpoint of the central thesis claim, and is reported as directionally non-significant on the available gold set of 14 CVs (Δ NDCG@10 = -0.047 , 95 per cent CI $[-0.168, +0.065]$), judged by Claude Sonnet 4.6. A post-hoc sensitivity check on the proposed calibration clamp shifts the headline to $\Delta = +0.018$ but does not change the inconclusive verdict.

6.2 Comparison with the initial objectives

I revisit the four objectives stated in Section 1.3 below.

Objective one — a working public service. The system runs on the open internet, supports user registration and login, accepts a CV upload as a PDF, fetches vacancies from seven external sources, and returns a personalised list with verdicts and bilingual explanations. The deployment is documented in Section 3.7 and Section 4.6. **Met.**

Objective two — a disciplined software architecture. The code base is organised as four .NET projects whose project-reference graph enforces the Clean Architecture dependency rule at build time. **Domain/Domain.csproj** declares zero package references; Application references only Domain; Infrastructure references Application (and Domain transitively); API references Application and Infrastructure. The SOLID principles are mapped to concrete artefacts in Section 3.3 and elaborated in Section 4.2, Section 4.3, and Section 4.4. **Met.**

Objective three — measurable match quality. The five-layer methodology in Section 5.2 is the architectural answer. The empirical answer is mixed, and honestly so. On

the reason-text layer the v6 prompt is a clear win over the v4 baseline across all four ordinal dimensions. On the ranking layer the central claim — that the Composite Judge improves ranking quality over the Linear baseline — is neither supported nor refuted at the standard significance level on 14 CVs; both pipelines reach strong full-ordering correlation against the cross-vendor relevance ratings (Spearman $\rho \approx 0.69$ each). The conditions under which the ranking result would resolve definitively are listed in Section 5.7 and Section 6.4. **Methodology met; empirical answer partial and openly characterised.**

Objective four — transparency of every LLM call. Every Gemini call writes a row to the `gemini_cost_log` table through the per-stage cost accumulator in `Application/Common/Diagnostics/CostBreakdown.cs`. The accumulator records the stage, the call count, the input and output token counts, the wall-clock duration, and the dollar cost computed from the current Gemini 2.5 Flash short-context tier prices. An admin endpoint exposes the data for offline querying. **Met.**

6.3 Encountered difficulties

In this section I record the four difficulties whose resolution shaped the chapters above. Each is documented in detail in the chapter cited.

Mid-thesis methodology pivot. My original primary endpoint was a single end-to-end score-MAE metric. Two audits showed it was structurally biased against any pipeline that deliberately departs from the deterministic calculator output (Section 5.4). I pivoted to top-ten NDCG against independent-vendor relevance ratings and kept the score-MAE as a diagnostic. The pivot is reported as a methodological contribution, not as a hidden failure.

Front-end and back-end contract drift. A back-end serialisation change in v6.7.8 (the addition of `JsonStringEnumConverter` at `API/Program.cs:66`) broke the front-end source filter and the verdict-label vocabulary at the same time. The silent empty-result failure and the two-file fix are documented in Section 4.7. The episode is the strongest argument for the single-source-of-truth maps on the front-end (Section 3.6).

Composite Judge calibration defect identified but not deployed. The rule-precedence override pattern that allows the Composite Judge to depart from its bounded-adjustment band is documented in Section 5.4, and the proposed fix is a single-file change at the boundary of `GeminiCompositeJudgeService`. I did not deploy the fix in time for the submitted evaluation. The post-hoc sensitivity check in Section 5.6 quantifies the defect’s contribution and confirms that the methodology’s verdict on the ranking layer holds whether the fix ships or not.

Single-vendor judge at submission time. The full multi-vendor judge ensemble — Claude plus an OpenAI judge plus a third-vendor judge — was constrained by API access at submission time. I adopted an intra-Anthropic configuration (Claude Opus 4.7 for reason-text, Claude Sonnet 4.6 for ranking) as a deliberate scope choice. The inter-vendor extension is recorded in Section 5.7 and below.

6.4 Future perspectives

In this section I name the extensions that would most directly tighten the empirical findings above. Each is a discrete piece of work with a clearly defined cost and a known location in the code or in the gold set.

Production deployment of the calibration clamp. The hard $|s_{\text{judge}} - s_{\text{linear}}| \leq 0.10$ clamp at the boundary of `GeminiCompositeJudgeService` is a single-file change. Deploying it requires re-running the ranking pipeline on the existing 14 CVs to produce a freshly clamped result, rather than the post-hoc sensitivity check that the chapter currently reports.

Multi-vendor judge ensemble. The single-vendor judge family is the principal limitation. Adding judges from one OpenAI model at its strongest tier and one model from a third vendor (for example Mistral’s Large tier) would let me measure inter-judge Cohen’s κ directly across vendor families. It would also show that the reason-quality and ranking-quality results are not artefacts of the Anthropic family.

Gold-set extension for per-family inference. The Layer 5 confidence interval of width roughly 0.23 reflects the $n = 14$ sample. Extending the CV gold set to at least 30 per role family (Engineering, Data, Product, Design) would support per-family inference and would tighten the aggregate interval enough to support a directional verdict.

Independent inter-annotator agreement. The match-quality gold ratings are produced by a single judge family and audited by me, the interested author. An inter-annotator coefficient on a stratified subset, computed by an annotator with no stake in the outcome, would establish the gold set’s external validity and would complement the judge self-consistency check in Section 5.5.

CI/CD pipeline. My current deployment is a manual but reproducible script (Section 4.6). A GitHub Actions workflow that builds the image, transfers it to the host, and runs the Compose roll-forward is straightforward to add and is the most concrete operational gap recorded in Section 4.6.

A/B production validation of the v6 prompt. The reason-quality result on v6 was obtained under the protocol of Zheng et al. and predicts user satisfaction by literature precedent; it does not measure user satisfaction directly. A v4-versus-v6 A/B comparison under real-user click-through and apply rates would close that gap.

Reference-free factuality evaluation as a Layer 4 complement. The MiniCheck factuality model of Tang et al. [9] is already integrated in the Infrastructure layer (Section 4.4) as `MLApiFactualityService` but is not evaluated in this thesis. Activating it as a third reason-quality method alongside rubric-based judging and programmatic checks would complement the multi-vendor ensemble with a reference-free claim-level verifier.

6.5 Final reflection

The thesis delivers a deployed system, a defensible layered evaluation methodology, a statistically significant positive empirical result on reason-text quality, and a directionally non-significant empirical result on ranking quality whose conditions for resolution I stated openly. The methodology stands regardless of the empirical verdict; the empirical

question on ranking quality remains open, with each of the extensions above proposed as the smallest piece of work that would meaningfully tighten the answer. Beyond the deliverables themselves, my main takeaway is the discipline of treating evaluation as a first-class engineering concern. The methodology pivot and the calibration audits in Chapter 5 were the hardest parts of the work, and they shaped the system more than any single architectural decision.

Glossary

Bibliography

- [1] S. Danziger, J. Levav, and L. Avnaim-Pesso, “Extraneous factors in judicial decisions,” *Proceedings of the National Academy of Sciences*, vol. 108, no. 17, pp. 6889–6892, 2011, doi: [10.1073/pnas.1018033108](https://doi.org/10.1073/pnas.1018033108).
- [2] R. C. Martin, *Clean Architecture: A Craftsman’s Guide to Software Structure and Design*. Prentice Hall, 2017.
- [3] Court of Justice of the European Union, “CV-Online Latvia SIA v. Melons SIA.” [Online]. Available: <https://curia.europa.eu/juris/document/document.jsf?docid=242047&doclang=EN>
- [4] G. Adomavicius and A. Tuzhilin, “Toward the next generation of recommender systems: A survey of the state-of-the-art and possible extensions,” *IEEE Transactions on Knowledge and Data Engineering*, vol. 17, no. 6, pp. 734–749, 2005, doi: [10.1109/TKDE.2005.99](https://doi.org/10.1109/TKDE.2005.99).
- [5] K. Järvelin and J. Kekäläinen, “Cumulated gain-based evaluation of IR techniques,” *ACM Transactions on Information Systems*, vol. 20, no. 4, pp. 422–446, 2002, doi: [10.1145/582415.582418](https://doi.org/10.1145/582415.582418).
- [6] P. Liang *et al.*, “Holistic Evaluation of Language Models,” *Transactions on Machine Learning Research*, 2023, Accessed: Jun. 03, 2026. [Online]. Available: <https://arxiv.org/abs/2211.09110>
- [7] L. Zheng *et al.*, “Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena,” in *Advances in Neural Information Processing Systems*, 2023. Accessed: Jun. 03, 2026. [Online]. Available: <https://arxiv.org/abs/2306.05685>
- [8] S. Hu and others, “AI Hiring with LLMs: A Context-Aware and Explainable Multi-Agent Framework.” Accessed: Jun. 04, 2026. [Online]. Available: <https://arxiv.org/abs/2504.02870>
- [9] L. Tang, P. Laban, and G. Durrett, “MiniCheck: Efficient Fact-Checking of LLMs on Grounding Documents,” in *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, Association for Computational Linguistics, 2024. [Online]. Available: <https://arxiv.org/abs/2404.10774>

A | Appendix

AA End-to-end worked example

In this appendix I take one CV and one vacancy, and run them through all five layers of the evaluation. The point is to show how the numbers actually work on a real pair. I picked this pair because the Composite pipeline wins on it clearly — exactly the kind of win the Composite Judge is meant to add. The aggregate result across all 14 CVs is still inconclusive (Section 5.6), but the layered methodology lets us see per-CV wins like this one instead of hiding them inside an average.

The pair I picked. The CV is **15_qa_senior_automation** — a senior QA automation engineer with almost nine years of experience, English at B2, and a stack of Python, Selenium, Playwright, Cypress, and JMeter. The vacancy is **e66e80c0** — a senior API and back-end performance/load testing role asking for exactly that Python + JMeter stack. I read the pair myself and put it at the top of the candidate set. The Sonnet judge agrees: it rates this pair 10/10 on **match_quality**.

Layer 1 — what the pipeline understood about the CV. From the gold normalisation:

- Seniority: senior
- English: B2
- Experience: about 107 months (8 years, 11 months)
- Skills: Python, Selenium, Playwright, Cypress, pytest, JMeter

The production pipeline reproduces these fields with roughly the same accuracy as the Layer 1 average reported in Section 5.3.

Layer 2 — what the pipeline understood about the vacancy. From the gold normalisation:

- Required seniority: senior
- Required English: B2
- Must-have skills: Python, API testing, JMeter, Gatling

The production normalisation reproduces this shape with roughly the same accuracy as the Layer 2 average in Section 5.3.

Layer 3 — the score. The Linear pipeline returns **0.642** (just the weighted sum of the seven sub-scores). The Composite pipeline returns **0.680** — the Composite Judge nudges the score up a little because the qualitative match is strong. The deterministic ideal from the gold inputs (Section 5.4) is in the same area. The small Linear → Composite step on this pair is exactly the kind of refinement the Judge is meant to make.

Layer 4 — the explanation the pipeline writes. The v6 prompt produces a bilingual reason text. It opens with “**Strong match: senior automation experience with the requested stack**”. Then three short sections:

- **Strengths:** Python, JMeter, API testing, Selenium (cross-context familiarity).
- **Gaps:** Gatling — but it sits close to JMeter, so it’s a minor gap.
- **Recommendation:** apply.

The Ukrainian version keeps brand names (Python, JMeter, Gatling, Selenium) in Latin script, as the **uk_term_preservation** rule asks. The Opus judge rates this text at or near the maximum on every rubric dimension.

Layer 5 — where the pipeline ranks the pair. I report two NDCG@10 numbers. **NDCG@10** is a ranking-quality metric that gives more weight to relevance at the top of the list than at the bottom (see §2.4). Δ is the difference between Composite NDCG@10 and Linear NDCG@10 — a positive Δ means Composite ranks better on this CV.

Like-for-like comparison. Only the 19 candidate pairs where both pipelines produced a clean score. (Composite had 11 scoring failures on the original 30-pair set.) Composite puts the chosen vacancy at **rank 5**; Linear puts it at **rank 8**. Composite NDCG@10 = **0.564**. Linear NDCG@10 = **0.442**. Per-CV Δ = +0.122.

Full coverage. Use every candidate each pipeline produced. Linear ranks the gold-ten vacancy at **rank 11** — outside the top ten — so its NDCG@10 drops to **0.230**. Composite NDCG@10 stays at **0.561**. Per-CV Δ = +0.331.

The two numbers measure different things: how the two pipelines compare on the same set, and how each pipeline handles its full set. The +0.331 figure is the one that feeds the aggregate reported in Section 5.6.

Composite rank	Composite score	Gold match_quality	Note
1	0.808	4	Composite ranks too high
2	0.780	8	
3	0.730	8	
4	0.730	8	
5	0.680	10	The pair we traced above
6	0.660	4	
7	0.630	4	
8	0.600	4	
9	0.588	4	
10	0.588	2	

Table 5: Composite top-ten ranking for the CV **15_qa_senior_automation** on the like-for-like subset. Our traced pair is at rank 5. The Linear pipeline puts the same pair at rank 8 on the same subset.

What this example shows. On this one pair, the Composite Judge does its job: it pushes a strongly relevant candidate higher than the deterministic calculator alone would. Layer 5 shows a clear positive Δ . The aggregate across all 14 CVs is still inconclusive — but an inconclusive aggregate does not mean every CV behaves the same way. The layered methodology shows us the per-CV picture instead of hiding it inside an average